

Degree Project in Cybersecurity

Second cycle, 30 credits

Bounded Contexts as Security Boundaries in Microservices

Authorization Enforcement Consistency

JAKUB RYBAK

Bounded Contexts as Security Boundaries in Microservices

Authorization Enforcement Consistency

JAKUB RYBAK

Date: June 13, 2026

Supervisor: Yacine Anser
Examiner: Mads Dam
School of Electrical Engineering and Computer Science
Swedish title: Bounded Contexts som säkerhetsgränser i mikrotjänster
Swedish subtitle: Konsekvent tillämpning av auktorisering

© 2026 Jakub Rybak

Abstract

Microservice architectures distribute access control across many independently deployed services, each of which must enforce authorization at its own boundary without a shared enforcement mechanism. Authorization checks at service boundaries are therefore often missing, inconsistently duplicated, or built on implicit trust in upstream services having already verified the caller, and these gaps are a recurring source of security weaknesses in deployed microservice systems. This thesis investigates whether aligning microservice boundaries with bounded contexts from Domain-Driven Design, while treating those boundaries as explicit security boundaries, improves the consistency with which authorization is enforced across services. Although bounded contexts are a widely adopted basis for microservice decomposition, their effect on authorization enforcement has received comparatively little systematic attention, and no established method exists for measuring such enforcement consistency as a structural property of a design. The investigation is conducted as a controlled design-level comparison of two architecture variants built around the same electronic commerce marketplace, the same four privileged operations, and the same attacker model, differing only in their boundary alignment strategy. Both variants are modeled using Data Flow Diagrams with explicit trust boundaries and analyzed using the STRIDE threat modeling framework, from which four metrics covering enforcement coverage, implicit-trust rate, authorization fragmentation, and verification depth are derived. The DDD-aligned variant achieves complete grounded enforcement coverage and eliminates implicit-trust flows, while the baseline variant leaves three of seven inter-service delivery points uncovered and three of seven flows under implicit trust. Three of four directional predictions are satisfied, yielding partial support for the hypothesis and a repeatable evaluation framework for further design-level analysis.

Keywords

Domain-Driven Design, Microservice Security, Threat Modeling, Authorization Consistency, Bounded Contexts, STRIDE, Data Flow Diagrams

Sammanfattning

Mikrotjänstarkitekturer fördelar åtkomstkontroll över många självständigt driftsatta tjänster, där varje tjänst måste genomdriva auktorisering vid sin egen gräns utan en gemensam mekanism. Auktoriseringskontroller vid tjänstegränser riskerar därför att saknas, dupliceras inkonsekvent eller bygga på implicit tillit till att uppströmstjänster redan verifierat anroparen, och dessa brister utgör en återkommande källa till säkerhetssvagheter i driftsatta mikrotjänstsystem. Detta examensarbete undersöker huruvida en anpassning av mikrotjänstgränser till avgränsade kontexter från domändriven design, där dessa gränser behandlas som explicita säkerhetsgränser, förbättrar konsistensen i auktoriseringens genomdrivande mellan tjänster. Även om avgränsade kontexter är en vedertagen grund för uppdelning av mikrotjänster har deras inverkan på auktoriseringens genomdrivande ägnats förhållandevis lite systematisk uppmärksamhet, och det saknas en etablerad metod för att mäta sådan konsistens som en strukturell egenskap av en arkitekturdesign. Undersökningen genomförs som en kontrollerad jämförelse på designnivå mellan två arkitekturvarianter utformade kring samma marknadsplats för elektronisk handel, samma fyra privilegierade operationer och samma angriparmodell, vilka skiljer sig endast i strategin för placering av gränser. Båda varianterna modelleras med dataflödesdiagram med explicit utritade förtroendegränser och analyseras med hotmodelleringsramverket STRIDE, varifrån fyra mått härleds som täcker täckningsgrad för genomdrivande, andel implicit tillit, fragmentering av auktorisering samt verifieringsdjup. Den domänanpassade varianten uppnår fullständig grundad täckning för genomdrivandet och eliminerar flöden baserade på implicit tillit, medan baslinjevarianten lämnar tre av sju leveranspunkter mellan tjänster utan täckning och tre av sju flöden under implicit tillit. Tre av fyra riktningsbestämda förutsägelser uppfylls, vilket utgör delvis stöd för hypotesen och tillhandahåller ett upprepningsbart utvärderingsramverk för fortsatt analys på designnivå.

Nyckelord

Domändriven design, Mikrotjänstsäkerhet, Hotmodellering, Auktoriseringskonsistens, Avgränsade kontexter, STRIDE, Dataflödesdiagram

Acknowledgments

I would like to thank Mads Dam and Yacine Anser for all their guidance and valuable feedback during the work on this thesis.

In accordance with the KTH guidelines on the use of generative AI in the master degree projects, generative AI was used during the writing of this thesis for language editing, i.e. improving spelling and phrasing, as well as for translating the abstract into Swedish.

Stockholm, June 2026
Jakub Rybak

Contents

1 Introduction **1**
1.1 Background . . . 2
1.2 Problem . . . 3
1.2.1 Original problem and definition . . . 3
1.2.2 Scientific and engineering issues . . . 3
1.3 Purpose . . . 4
1.4 Goals . . . 5
1.5 Research Methodology . . . 5
1.6 Delimitations . . . 6
1.7 Structure of the thesis . . . 7

2 Background **9**
2.1 Domain-Driven Design and Microservice Decomposition . . . 9
2.1.1 The Domain Model . . . 9
2.1.2 Bounded Contexts and Context Maps . . . 10
2.1.3 Applying DDD to Microservice Decomposition . . . 11
2.2 Microservice Security and Authorization . . . 12
2.2.1 The Microservice Security Problem . . . 12
2.2.2 Authorization Patterns and Defense-in-Depth . . . 13
2.2.3 Attack Surface and Boundary Exposure . . . 15
2.3 Threat Modeling with DFDs and STRIDE . . . 16
2.3.1 Data Flow Diagrams . . . 16
2.3.2 The STRIDE Framework . . . 17
2.3.3 Design-Level Application . . . 18
2.4 Related Work . . . 19
2.4.1 Security in Microservice Architectures . . . 19
2.4.2 DDD-Aligned Decomposition in Practice . . . 20
2.4.3 Zero Trust Architecture . . . 20
2.4.4 Research Gap . . . 21

3 Method **23**
3.1 Research Process Overview . . . 23
3.2 Research Paradigm . . . 25
3.3 Comparative Study Design . . . 26
3.3.1 Controlled Parameters . . . 26
3.3.2 Independent and Dependent Variables . . . 27
3.3.3 Attacker Model . . . 27
3.3.4 DFD Modeling Conventions . . . 28
3.3.5 STRIDE Scope and Category Justification . . . 29
3.3.6 Design-Level Comparison Rationale . . . 30
3.4 Alternative Methodologies . . . 31
3.4.1 Runtime and Implementation-Based Testing . . . 31
3.4.2 Systematic Literature Review or Expert Survey . . . 31
3.4.3 Solution-Aware DFDs . . . 31
3.5 Evaluation Framework . . . 32
3.5.1 M1: Grounded Enforcement Coverage . . . 34
3.5.2 M2: Normalized Implicit-Trust Rate . . . 34
3.5.3 M3: Authorization Fragmentation Score . . . 35
3.5.4 M4: Cross-boundary Verification Depth . . . 36
3.5.5 Flow Classification Rules . . . 37
3.6 Assessing Reliability and Validity . . . 39
3.6.1 Construct validity . . . 39
3.6.2 Reproducibility . . . 40
3.7 Software and Modeling Tools . . . 41
3.8 Summary . . . 42

4 Threat Modeling **43**
4.1 Domain and Privileged Operations . . . 43
4.2 The DDD-Aligned Variant . . . 46
4.2.1 Service Decomposition . . . 46
4.2.2 System-Level DFD . . . 47
4.2.3 STRIDE Threat Analysis . . . 48
4.2.3.1 OP-1: Seller Listing Publication . . . 49
4.2.3.2 OP-2: Order Placement with Payment Capture . . . 50
4.2.3.3 OP-3: Return Request and Refund Authorization . . . 52
4.2.3.4 OP-4: Administrative Dispute Override . . . 55
4.3 The Baseline Variant . . . 57

4.3.1 Service Decomposition . . . 57
4.3.2 Data Flow Diagram . . . 59
4.3.3 STRIDE Threat Analysis . . . 61
4.3.3.1 OP-1: Seller Listing Publication . . . 62
4.3.3.2 OP-2: Order Placement with Payment Capture . . . 64
4.3.3.3 OP-3: Return Request and Refund Authorization . . . 66
4.3.3.4 OP-4: Administrative Dispute Override . . . 68
4.3.4 Summary . . . 69

5 Results and Analysis **71**
5.1 Structural Overview of Both Variants . . . 71
5.2 Per-Operation Metric Results . . . 72
5.2.1 OP-1: Seller Listing Publication . . . 72
5.2.2 OP-2: Order Placement with Payment Capture . . . 73
5.2.3 OP-3: Return Request and Refund Authorization . . . 75
5.2.4 OP-4: Administrative Dispute Override . . . 76
5.3 Aggregate Metric Results . . . 78
5.3.1 M1: Grounded Enforcement Coverage . . . 78
5.3.2 M2: Normalized Implicit-Trust Rate . . . 79
5.3.3 M3: Authorization Fragmentation Score . . . 80
5.3.4 M4: Cross-Boundary Verification Depth . . . 82
5.4 Relation to Prior Work . . . 83
5.5 Hypothesis Assessment . . . 84

6 Conclusions and Future work **87**
6.1 Conclusions . . . 87
6.2 Limitations . . . 89
6.3 Future work . . . 91
6.4 Reflections . . . 92

References **95**

List of Figures

3.1 Overview of the research process. 24

4.1 System-level DFD for the DDD-aligned variant. 48
4.2 Operation-level DFD for OP-1 in the DDD-aligned variant. . . 49
4.3 Operation-level DFD for OP-2 in the DDD-aligned variant. . . 50
4.4 Operation-level DFD for OP-3 in the DDD-aligned variant. . . 52
4.5 Operation-level DFD for OP-4 in the DDD-aligned variant. . . 55
4.6 System-level DFD for the baseline variant. 60
4.7 Operation-level DFD for OP-1 in the baseline variant. 62
4.8 Operation-level DFD for OP-2 in the baseline variant. 64
4.9 Operation-level DFD for OP-3 in the baseline variant. 66
4.10 Operation-level DFD for OP-4 in the baseline variant. 68

List of Tables

3.1 Evaluation metric definitions. 33
3.2 Flow classification under the evaluation framework. A dash indicates the flow class does not contribute to that metric. . . . 38

4.1 Privileged operation selection. 44
4.2 Summary of the architecture variants. 45
4.3 Summary of DFD flow identifiers and operation mapping. . . 45
4.4 STRIDE for OP-1 in the DDD-aligned variant. 50
4.5 STRIDE for OP-2 in the DDD-aligned variant. 52
4.6 STRIDE for OP-3 in the DDD-aligned variant. 54
4.7 STRIDE for OP-4 in the DDD-aligned variant. 57
4.8 STRIDE for OP-1 in the baseline variant. 63
4.9 STRIDE for OP-2 in the baseline variant. 65
4.10 STRIDE for OP-3 in the baseline variant. 67
4.11 STRIDE for OP-4 in the baseline variant. 69

5.1 Structural topology summary of both architecture variants. . . 71
5.2 Flow classification for OP-1 (per Table 3.2). 73
5.3 Metric input quantities for OP-1. 73
5.4 Flow classification for OP-2 (per Table 3.2). 74
5.5 Metric input quantities for OP-2. 75
5.6 Flow classification for OP-3 (per Table 3.2). 75
5.7 Metric input quantities for OP-3. 76
5.8 Flow classification for OP-4 (per Table 3.2). 76
5.9 Metric input quantities for OP-4. 77
5.10 Aggregated metric values for both architecture variants. 78

List of acronyms and abbreviations

API	Application Programming Interface
CVE	Common Vulnerability and Exposure
DDD	Domain-Driven Design
DFD	Data Flow Diagram
JWT	JSON Web Token
mTLS	mutual Transport Layer Security
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
PAP	Policy Administration Point
PDP	Policy Decision Point
PEP	Policy Enforcement Point
PIP	Policy Information Point
RBAC	Role-Based Access Control
SDL	Security Development Lifecycle
SQL	Structured Query Language
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege
XSS	Cross-Site Scripting

Chapter 1

Introduction

Microservice architectures decompose a software system into a set of independently deployable services, each responsible for a focused slice of system functionality. This decomposition brings well-known benefits, including independent scalability and organizational alignment between teams and services [1]. However, it also distributes the security surface across many components that must collectively enforce consistent access control, and the risk of inconsistent enforcement grows with the number of services and the complexity of their interactions [2], [3]. In systems designed using monolithic architecture, authentication and authorization can be handled at a single shared perimeter, while in microservice systems, they must be applied at each service boundary, which in turn creates new categories of architectural security problems [2], [3].

This thesis investigates whether aligning microservice decomposition with the bounded context model of Domain-Driven Design (DDD), while treating those boundaries as explicit security boundaries, can improve the consistency with which authorization is enforced across microservices. The rationale is that a bounded context, by delimiting a coherent set of domain responsibilities within a single deployable boundary, creates the structural conditions under which the authorization decisions relevant to that domain can be co-located with the logic they protect, rather than scattered across services that share no common ownership of the domain [2], [4]. Authorization enforcement consistency, as used throughout this thesis, refers to the degree to which every privileged operation is protected by an explicit authorization check at every trust-boundary crossing it traverses, without reliance on upstream services having already verified the access rights of the caller. It is assessed through four design-level metrics derived from the Data Flow Diagram

(DFD) and Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege (STRIDE) models, specifically enforcement coverage, implicit-trust rate, authorization fragmentation, and cross-boundary verification depth, as defined in full in Section 3.5 [2], [3]. The investigation is structured as a controlled, design-level comparative study of two architecture variants designed around the same e-commerce marketplace requirements, evaluated through threat modeling using DFDs and the STRIDE framework.

1.1 Background

Authorization-related weaknesses are among the most frequently identified security flaws in deployed microservice architectures, which has been confirmed both by targeted vulnerability analyses and by systematic literature reviews of the field [5], [6]. The architectural cause is generally well understood, as independent service deployment forces each service to enforce its own authorization policy without any shared enforcement mechanism to guarantee consistency across the system [2], [3]. Consequently, authorization checks at some of the boundary crossings may be missing, inconsistently duplicated, or built with reliance on implicit trust in upstream claims rather than on explicit internal verification [2].

DDD is a software engineering approach introduced by Eric Evans that organizes system design around a domain model that represents business responsibilities [4]. Moreover, a central concept in DDD is a bounded context, which Evans defines as an explicit boundary, often corresponding to a subsystem, within which a particular model is defined, applicable, and kept internally consistent [4]. Bounded contexts have been found to provide a practical and widely adopted basis for microservice decomposition, since aligning service boundaries with domain responsibilities reduces coupling and improves the architectural clarity [7]. However, the question whether those boundaries can also serve as security boundaries, and what effect this alignment has on authorization enforcement consistency, has received comparatively little systematic attention in the literature [6].

The approach to threat modeling used in this thesis is based on two complementary techniques. DFDs provide a structured notation for representing system components, flow of data between them, and trust boundaries at the design level. In particular, solution-aware DFDs have been shown to support security analysis without requiring a running implementation [8]. STRIDE is a threat classification framework that

organizes threats into six categories and has been applied systematically in industrial threat modeling practice at scale [9]. The combination of DFDs and STRIDE has been used in recent work to analyze the security of software systems at the design level, providing a structured and repeatable framework for enumerating threats across modeled components and data flows [10], [11].

1.2 Problem

1.2.1 Original problem and definition

In a microservice-based system, each service is deployed independently, maintains its own data store, and communicates with other services through well-defined interfaces. This means that any privileged operation that requires access control must be enforced within each of the participating services, rather than at a shared enforcement point. Moreover, when service boundaries are not designed with security responsibilities in mind, authorization checks at boundary crossings may be missing or inconsistently enforced across services [2], [3].

This often results in an architecture where the overall authorization enforcement strategy is difficult to reason about, audit, or consistently maintain as the system grows. DDD proposes a decomposition model, in which each bounded context owns a specific set of domain responsibilities and exposes well-defined interfaces for communication between the contexts [4], [7]. The working hypothesis is that DDD-aligned bounded contexts lead to more consistent authorization enforcement because security decisions remain co-located with domain responsibility.

Therefore, the central question is whether this co-location of domain and security responsibilities measurably improves the authorization enforcement consistency, compared to an architecture that does not apply DDD-aligned decomposition. This leads to the following research question:

RQ. How does aligning microservice boundaries with Domain-Driven Design bounded contexts that encapsulate security responsibilities affect the consistency of authorization enforcement across microservices?

1.2.2 Scientific and engineering issues

The scientific challenge in this investigation lies in successfully operationalizing authorization enforcement consistency as a measurable property of

an architecture, which exists only as a design model without a running implementation, from which runtime behavior can be observed. Therefore, the evaluation must rely on model-derived metrics applied to DFDs with explicitly drawn trust boundaries. This requires both a precise definition of the metrics and a controlled modeling procedure that ensures the two compared variants are equivalent in all parameters except the strategy of the boundary alignment itself.

The engineering challenge, on the other hand, lies in designing two architecture variants, one DDD-aligned and one baseline, that implement the same e-commerce marketplace requirements with a controlled and comparable level of granularity. Achieving this requires an explicit definition of the attacker model, a fixed set of privileged operations applied symmetrically to both variants, and a consistent STRIDE procedure applied to both DFDs [9]. Consequently, any differences measured in the evaluation metrics must be attributable to the boundary alignment strategy rather than to differences in modeling scope or level of completeness.

1.3 Purpose

The purpose of this thesis is to produce a structured, design-level analysis that determines whether treating DDD bounded contexts as security boundaries improves the consistency of authorization enforcement in microservice architectures, and to contribute model-based evidence to the intersection of software architecture and security engineering. The intended audience of this research includes software architects and security engineers who face design decisions related to decomposition in microservice systems, as well as researchers working at the intersection of domain-driven design and cybersecurity [6]. Finally, by providing a repeatable model comparison method grounded in established threat modeling techniques, the thesis contributes a specific evaluation framework that practitioners can adapt to their own systems.

From an ethical and societal perspective, the work contributes indirectly by supporting the process of making more informed and verifiable architectural decisions. This in turn reduces the future likelihood of authorization failures that lead to unauthorized access to sensitive user data or privileged system operations.

1.4 Goals

The primary goal of this degree project is to design and document two comparable microservice architecture variants implementing the same e-commerce marketplace requirements, one following DDD-aligned decomposition where bounded contexts serve as explicit security boundaries, and one following a baseline decomposition that does not apply domain-aligned boundary placement. The authorization enforcement consistency of both variants is then evaluated through systematic threat modeling.

The secondary goal is to define and apply a set of model-derived metrics that quantify authorization enforcement consistency at the design level, and to interpret the metric differences between the two variants in relation to the research question and hypothesis.

Consequently, the concrete deliverables of the project are a defined marketplace domain scope covering four functional areas and four privileged operations, instantiated as two distinct service decompositions reflecting different boundary alignment strategies. Furthermore, for both architecture variants, specific deliverables are DFDs with explicit trust boundaries, STRIDE threat enumeration scoped to authorization-relevant threats, computed evaluation metrics, and a written analysis comparing and interpreting the results.

1.5 Research Methodology

The research methodology adopted in this thesis is a design-level comparative analysis, combining both qualitative and quantitative elements. The qualitative component consists of the structured design and documentation of the two distinct architecture variants, together with the systematic application of STRIDE to identify authorization-relevant threats in each. The quantitative component consists of the collection and comparison of model-derived metrics that measure authorization enforcement consistency across the two designs [12], [13].

This mixed approach is appropriate because it produces structured, repeatable, and traceable results at the design level without requiring a full running implementation or empirical security testing [2]. The underlying assumption is that security properties of a software system can be meaningfully analyzed and compared through formal design-level modeling, provided the models are constructed with sufficient precision and controlled

comparability. This assumption is consistent with the established use of DFDs and STRIDE as tools for security analysis at the design level in both academia and industry [8], [9], [10].

The research approach is deductive, a hypothesis is stated, two contrasting architectural variants are constructed under fixed and identical assumptions, and the hypothesis is evaluated against the metric evidence collected from the models. The two variants share the same e-commerce marketplace domain scope, the same attacker model (an authenticated low-privilege user), the same set of privileged operations, and the same assumptions regarding identity and authentication infrastructure. STRIDE is applied using the same procedure for both variants, with the scope restricted to authorization-relevant threat categories at trust-boundary crossings and flows involving privileged operations [9]. The methodology is described in full in Chapter 3.

1.6 Delimitations

This thesis is limited to a design-level analysis and does not include the implementation, deployment, or practical testing of either architecture variant. All findings are therefore derived from the DFD models and the STRIDE threat enumeration applied to those models. As a result, claims about runtime behavior or deployed system performance are outside the scope of this work.

Additionally, the threat analysis is scoped exclusively to authorization-relevant threats under the defined attacker model of an authenticated low-privilege user attempting a fixed set of privileged operations. General implementation-level vulnerabilities, such as Structured Query Language (SQL) injection, Cross-Site Scripting (XSS), or third-party library Common Vulnerabilities and Exposures (CVEs), are considered out of scope unless they directly enable an authorization bypass that can be represented at the design level [9]. Threats related to components that do not participate in the authorization flows of the selected privileged operations, such as external integrations, as well as monitoring or logging infrastructure, are similarly excluded.

Lastly, the domain context is restricted to an e-commerce marketplace with a controlled level of complexity, covering 4 functional areas and 4 privileged operations instantiated across two distinct service decompositions. This restriction is necessary to keep the analysis feasible and to maintain a controlled comparison between the two variants. Consequently, the applicability of the findings to other domains, larger systems, or systems with different authentication infrastructure is limited, and generalizations beyond

the stated scope should be made with caution.

1.7 Structure of the thesis

Chapter 2 presents the background on DDD and the bounded context concept, microservice security and authorization patterns, as well as threat modeling using DFDs and STRIDE, and concludes with a discussion of related work and the research gap this thesis addresses. Chapter 3 describes the research methodology and the specific methods used to conduct the comparative analysis. Chapter 4 documents the design of the two architecture variants, the construction of the DFDs, and the application of STRIDE. Chapter 5 presents and analyzes the results of the threat modeling, as well as discusses the findings from the comparison in relation to the research question and hypothesis. Chapter 6 focuses on drawing the conclusions and outlines directions for possible future work.

Chapter 2

Background

This chapter presents the background knowledge required to understand the investigation conducted in this thesis. The first part covers DDD, its central concepts, as well as its established use as a decomposition basis for microservice architectures. The second part focuses on the security challenges specific to microservice systems, with emphasis on authorization enforcement and the architectural patterns used to address it. The third part introduces the threat modeling methodology used in this thesis, covering DFDs and the STRIDE framework. Finally, the chapter concludes with a review of related work and an identification of the research gap that this thesis addresses.

2.1 Domain-Driven Design and Microservice Decomposition

2.1.1 The Domain Model

DDD is a software engineering approach in which the structure and logic of a software system are organized around a model of the business domain that it represents [4]. The domain model is the central element of this approach that captures the concepts, complex relationships, and business rules of the domain in a form that both developers and domain experts can directly work with [4]. Rather than treating the domain model as a documentation artifact separate from the implementation, DDD requires that the model and the underlying code remain aligned, so that changes to the understanding of the business domain are reflected immediately in the software [4].

This alignment is enabled by the ubiquitous language, a shared vocabulary structured around the domain model that is used across conversation,

documentation, and code, which ensures the model is expressed directly in the implementation and prevents translation errors between developers and domain experts [4]. On the technical side, the domain layer is separated from infrastructure and application concerns, and domain objects are organized into aggregates, which are clusters of related entities accessed and modified exclusively through a single aggregate root that enforces the domain's internal consistency rules [4].

2.1.2 Bounded Contexts and Context Maps

As a domain model grows, it becomes increasingly difficult to maintain a single unified domain model across all parts of a large system. Evans observes that total unification of the domain model for a large enterprise system is neither feasible nor cost-effective, and that multiple models are inevitable to emerge in different parts of such system [4]. Therefore, the challenge is not to prevent this divergence directly, but rather to manage it consciously so that model inconsistencies do not produce unreliable software.

The bounded context is the primary strategic pattern for managing this situation. Evans defines a bounded context as the explicit boundary, within which a particular domain model applies and is kept strictly consistent, while different models, with potentially different terminology and rules, apply in neighboring contexts [4]. Moreover, such boundary is defined not only in pure conceptual terms, but also in terms of concrete elements, specifically team organization, the parts of the application that use the model, and physical implementations such as code bases and database schemas [4]. Accordingly, a bounded context delimits the scope within which a given ubiquitous language is valid.

However, a bounded context is not the same as a software module, since modules organize elements within a single domain model, whereas bounded contexts separate distinct domain models from one another [4]. Teams working within a bounded context maintain one consistent domain model by regularly aligning their concepts and code to prevent divergences [4]. This explicit boundary makes local consistency achievable, as the team knows exactly what must remain consistent, as opposed to what belongs to a different context with different rules.

The relationships between bounded contexts are captured in a context map, which Evans describes as a global representation of the project's contexts and the relationships between them, making it explicit whether each inter-context exchange involves translation, shared ownership, or one-sided adaptation [4].

One pattern of particular relevance to this thesis is the Anti-Corruption Layer, in which a context protects its own model by placing a translation layer at an incoming boundary, which also serves as a natural enforcement point for authorization decisions at that boundary crossing [4].

For this thesis, the bounded context boundary is the primary unit of security analysis, since it represents the scope within which a single, coherent set of domain rules can be defined and maintained, and therefore also the scope within which the authorization decisions governing operations on that domain can be consistently located and enforced.

2.1.3 Applying DDD to Microservice Decomposition

The bounded context concept provides a natural decomposition basis for microservice architectures, and this connection has been widely adopted in both industry and academic work [7], [14], [15]. In a microservice architecture, each service is expected to be autonomous, maintain its own data store, and to expose well-defined interfaces to other services. These properties correspond directly to what a bounded context guarantees, namely a clearly delimited scope of applicability, internal consistency, and explicit interface contracts for communication with neighboring contexts [7].

Hippchen et al. describe a structured development process for building microservice-based applications using DDD, in which the domain model is first constructed as an information model during collaborative modeling with domain experts, then divided into bounded contexts based on the organizational structure of the domain, and finally refined and extended by each responsible team [7]. The resulting context map guides the decomposition of the system into individually deployable services, where each service corresponds to one bounded context and owns the data and business logic that belong to it [7]. Furthermore, the decomposition process also includes systematic derivation of web Application Programming Interfaces (APIs) from the domain model, ensuring that each service's interface reflects the ubiquitous language of its bounded context and does not expose internal domain objects directly [7].

The structural benefits that DDD-aligned decomposition produces, specifically stable interfaces, reduced coupling, and clear domain ownership, are relevant to this thesis not as ends in themselves, but because they create the architectural preconditions under which authorization enforcement can also be aligned with domain boundaries, a connection that the threat modeling comparison in Chapters 4 and 5 examines directly [4].

2.2 Microservice Security and Authorization

2.2.1 The Microservice Security Problem

Microservice architectures introduce a range of security problems that differ structurally from those commonly found in monolithic systems. In a monolithic system, authentication and authorization can be enforced at a single shared boundary, whereas in a microservice system, every service is independently deployed and must enforce its own access control rules, since there is no shared runtime that could apply a single enforcement policy across all components [2], [3], [6]. Consequently, security is no longer a centralized concern, but rather becomes a distributed coordination problem, where consistent enforcement depends on each of the participating services behaving correctly.

Systematic literature reviews confirm that this structural challenge emerges in practice as a persistent and frequent source of security flaws. Berardi et al., in a review of 290 peer-reviewed publications on microservice security, identify access control as one of the leading vulnerability categories across deployed microservice systems. Moreover, they also note that architectural-level security, as opposed to communication-level or implementation-level security, in comparison receives little attention in the literature [6]. Furthermore, Jayalath et al. provide a more granular categorization, classifying authorization and policy enforcement vulnerabilities under categories that include missing permission checks, inconsistent role-based access control, as well as improper propagation of identity context across service calls [5]. Moreover, Hutasuhut et al. confirm in a 2024 systematic review that access control remains the most critical unresolved security concern in microservice architectures, while making a point that generic security-by-design models for such architectures are generally absent from the existing literature [16].

The specific form of the authorization problem in microservice systems arises from the combination of service independence and the need for cross-service interactions. In particular, when a client request triggers a chain of service-to-service calls, each participating service must verify independently that the caller is authorized to perform the requested operation, rather than relying on a prior identity check performed by an upstream service [2], [3]. Reliance on implicit upstream trust, where a downstream service assumes that an upstream service has already validated authorization, creates vulnerability to lateral movement attacks, in which a compromised

or misconfigured upstream service forwards unauthorized requests that are accepted without question [2], [17], [18]. Furthermore, the absence of a shared enforcement point means that adding a new service, changing existing service's responsibilities, or modifying a cross-service flow can silently introduce authorization gaps that are difficult to detect without a complete audit of all affected flows [3].

Reliance on implicit upstream trust reproduces, at the level of individual services, the assumption that perimeter-based network security makes at the level of an entire system, namely that a single check at the point of entry is sufficient and that everything behind that point can be trusted by default. Industry examples document how this assumption fails in practice. He et al. recount that Google, following a sophisticated targeted intrusion in 2009, abandoned the assumption of a trusted internal network entirely and rebuilt its access architecture into the BeyondCorp model, in which every access to an internal application is authenticated and explicitly authorized regardless of whether the user or device resides inside or outside the corporate network [19]. The same survey cites the 2015 cyberattack on the Ukrainian power grid, which caused a large-scale blackout, as an example of the damage a successful intrusion can inflict on critical infrastructure [19]. Although these episodes concern enterprise and infrastructure networks rather than microservice systems specifically, the structural lesson of the former, that default trust extended behind a single entry check is exploitable, is the same condition that this thesis examines at the architectural level, namely components that accept requests under implicit trust because an upstream check is assumed to have already been performed.

The thesis investigates whether deliberately aligning service boundaries with domain responsibilities, as DDD proposes, reduces the conditions under which these gaps arise by ensuring that each service owns a coherent and bounded set of authorization decisions, rather than sharing enforcement responsibility across arbitrarily drawn boundaries [2], [3].

2.2.2 Authorization Patterns and Defense-in-Depth

Addressing the distributed authorization problem in microservice architectures requires a structured approach to decisions concerning placement and management of the enforcement logic. The Open Worldwide Application Security Project (OWASP) Microservices Security Cheat Sheet describes the authorization pattern model in terms of four roles, consistent with National Institute of Standards and Technology (NIST) SP 800-162 specification, the

Policy Administration Point (PAP), which manages the creation and storage of authorization policies, the Policy Decision Point (PDP), which evaluates an access request against the applicable policy, the Policy Enforcement Point (PEP), which intercepts the request and enforces the decision, and finally the Policy Information Point (PIP), which provides the PDP with the contextual data required to evaluate the policy [2], [20]. In the context of this thesis, each service is responsible for evaluating the authorization claims carried in the incoming request at its own boundary, with identity context supplied by a shared identity infrastructure [2], [3].

NIST SP 800-204 reinforces the principle that authorization enforcement must occur at the individual service level, not only at the API gateway. Chandramouli specifically identifies as a security strategy the requirement for fine-grained authorization at the microservice, distinct from and in addition to any coarse-grained filtering at the gateway, because the gateway itself cannot, and should not, possess the business-domain context required to evaluate fine-grained access rules [3]. This three-layer defense-in-depth model, in which authorization is applied at the edge (API gateway), at the service boundary, and within the business logic of each service, is described by OWASP as the recommended enforcement architecture [2], [3]. Therefore, a microservice architecture in which some services enforce authorization only at the gateway level, rather than also locally, creates a structural gap that an attacker can exploit by bypassing or compromising the gateway.

Service-to-service authentication and the propagation of identity context are closely related concerns. When a service receives a request from another service rather than directly from a client, it must authenticate the calling service, verify that the calling service is authorized to perform the requested operation, and ensure that the end-user identity and associated claims are propagated correctly through the call chain [2]. Mechanisms such as mutual Transport Layer Security (mTLS) address service-to-service authentication at the transport layer, while token-based standards such as OAuth 2.0 and JSON Web Token (JWT) address the propagation of user identity and claims through the call chain [2], [21], [22], [23]. Leines-Vite et al. provide a mapping from STRIDE threat categories to specific security mechanisms that address them in microservice systems, noting that spoofing threats are addressed by mTLS and token-based authentication, tampering threats by message integrity mechanisms, and elevation of privilege threats by Role-Based Access Control (RBAC) and fine-grained authorization enforcement [23].

De Almeida and Canedo, in a systematic literature review of authentication and authorization in microservice architectures, identify the top challenges

as managing communications between microservices securely, implementing fine-grained access control per service, propagating identity context across service boundaries, as well as avoiding single points of failure in the authorization infrastructure [24]. Their review of 24 primary studies further confirms that no single mechanism simultaneously addresses all of these challenges, and that combinations of OAuth 2.0, JWT, API gateways, and service-level authorization logic are the most commonly recommended approaches [24].

2.2.3 Attack Surface and Boundary Exposure

The attack surface of a system is the set of entry points through which an attacker can attempt to interact with it, consisting of all interfaces, data flows, and trust boundaries that are exposed to untrusted input [25], [26]. In microservice architectures, the attack surface is structurally larger than in monolithic systems, because each independently deployed service exposes its own interfaces and each service-to-service communication channel represents an additional potential attack path [26]. Therefore, reducing the attack surface, through minimizing the number of exposed interfaces, restricting which services can call which, and ensuring that each interface is explicitly scoped to the operations it needs to support, is a fundamental security design principle in this context.

The empirical evidence from the microservice security literature confirms that a poorly scoped or poorly bounded service is a persistent source of authorization vulnerabilities. Jayalath et al. document specific vulnerability categories that arise directly from inadequate boundary definition, including overly broad service responsibilities that force services to accept requests involving privilege levels they cannot evaluate correctly, as well as missing cross-service authorization checks that occur when a new call path crosses a boundary without a corresponding enforcement update [5]. Furthermore, Berardi et al. identify the lack of clear trust boundaries between services as a structural precondition for a range of authorization and access control vulnerabilities, while noting that the problem is rarely addressed at the architectural design stage [6].

The relationship between service boundary design and authorization consistency motivates the central investigation of this thesis. When service boundaries are drawn to reflect domain responsibilities, each service is responsible for a coherent and well-defined set of operations, and the authorization enforcement rules governing those operations can be located

together with the domain logic they protect. On the other hand, when boundaries are drawn for other reasons, such as technical convenience or organizational accident, the correspondence between service scope and authorization responsibility can often become unclear, and the risk of gaps and inconsistencies might increase. This connection between boundary design and authorization posture is examined in detail through the comparative threat modeling analysis in Chapters 4 and 5.

2.3 Threat Modeling with DFDs and STRIDE

2.3.1 Data Flow Diagrams

A DFD is a visual notation for representing the flow of data through a system, the components that process or store that data, as well as the external entities that interact with it. In the context of security threat modeling, DFDs serve as the primary initial representation for systematic threat enumeration, where the diagram captures the system architecture at a level of abstraction that exposes the security-relevant structure without requiring a running implementation [8]. Furthermore, the notation uses four element types, an external entity, representing a person or system outside the control of the modeled system, a process, representing a component that transforms or routes data, a data store, representing persistent data at rest, and finally a data flow, representing the movement of data between individual elements [9].

Sion et al. provide a formal meta-model for DFDs that makes the notation precise enough to support automated threat identification. In their meta-model, each element type is a distinct meta-class, such as a DataFlowEntity (the superclass of Process, DataStore, and ExternalEntity), DataFlow (connecting a sender to a recipient), and TrustBoundaryContainer (grouping elements that operate at the same privilege level) [8]. Trust boundaries, represented as dashed lines or boxes in the diagram, define where different parts of the system operate at different privilege levels, with many threat categories applying specifically to data flows that cross these boundaries [8], [9]. Consequently, the explicit placement of trust boundaries in a DFD is not a purely aesthetic choice, but rather determines which flows are evaluated against which threat categories during the analysis.

However, a standard DFD representation does not capture information about the security solutions already applied to the system, which in turn can lead to enumeration of many threats that are already mitigated by known mechanisms. Sion et al. address this by proposing solution-aware DFD, which

enrich the standard notation with security solution annotations that link DFD elements to roles in a catalog of security solutions, making mitigations already present in an architecture visible within the model [8]. For the purposes of this thesis, DFDs are used at the design level to represent the two architecture variants, and trust boundaries are placed explicitly at every point where a service-to-service interaction crosses a bounded context boundary.

2.3.2 The STRIDE Framework

STRIDE is a threat classification framework developed at Microsoft and used as the core of the Security Development Lifecycle (SDL) threat modeling methodology. The name is an acronym for six threat categories, Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege [9]. Each of the categories corresponds to a violation of a specific security property, Spoofing violates authentication, Tampering violates integrity, Repudiation violates non-repudiation, Information Disclosure violates confidentiality, Denial of Service violates availability, and finally Elevation of Privilege violates authorization [9], [11].

The STRIDE methodology as practiced at Microsoft follows a four-step SDL process, starting with modeling the system using DFDs with trust boundaries, followed by enumerating threats systematically using STRIDE-per-element, then identifying mitigations for each of the threats, and finally validating the threat model against the final design [9], [27]. The STRIDE-per-element procedure is the key innovation, where rather than relying on brainstorming, which produces results that are neither repeatable nor complete, the analyst iterates over each element in the DFD and applies a defined subset of STRIDE categories based on type of the element [9], [28]. Shostack observes that this approach was specifically designed to allow engineers without deep security expertise to perform credible threat modeling, because the enumeration is guided by the structure of the diagram rather than by the analyst's prior knowledge of possible attack patterns [9].

Khan et al. extend the STRIDE-per-element procedure by specifying explicitly which STRIDE categories are applicable to each DFD element type and which to each interaction type. In their formulation, external entities are subject to Spoofing, data flows are subject to Tampering, Information Disclosure, and Denial of Service, processes are subject to all six STRIDE categories, and finally data stores are subject to Tampering, Information Disclosure, Denial of Service, and Elevation of Privilege [11]. This mapping

provides a systematic and reproducible enumeration procedure that ensures no element type is overlooked during the analysis. Furthermore, Khan et al. demonstrate the procedure through a practical application to a Cyber-Physical System, showing that the resulting threat set is comprehensive relative to the system's DFD structure and that it can be linked directly to possible mitigation strategies [11].

2.3.3 Design-Level Application

A fundamental property of the methodology combining STRIDE and DFD is that it operates on a model of the system rather than on the system itself, which makes it applicable at the design stage before any implementation exists. Sion et al. establish this property formally by showing that DFDs are architectural representations that capture the security-relevant structure of a system in a form that is both accessible and sufficient for threat identification [8]. The methodology therefore supports security-by-design reasoning, where rather than discovering threats through penetration testing or code review after the system is built, the analyst identifies them from the DFD and addresses them as design decisions [8], [9].

The scope of a design-level STRIDE analysis can be precisely controlled by restricting it to specific DFD elements and flows. Shostack describes the SDL approach of scoping the analysis to the elements and flows that are most relevant to the investigated security properties, rather than exhaustively analyzing every element in a complete system diagram [9]. In this thesis, the scope is restricted to the DFD elements and flows that participate in the authorization of the selected privileged operations, specifically the trust-boundary crossings and the flows that carry or consume identity and authorization context. As such, threats that do not affect authorization enforcement under the defined attacker model are explicitly excluded.

The design-level approach carries a well-understood limitation, namely it cannot detect implementation-specific vulnerabilities that are not representable at the model level, such as SQL injection or cryptographic implementation errors [9]. However, for the purpose of this thesis, comparing two architecture variants at the boundary-design level, this limitation is appropriate rather than problematic, since both variants are subject to the same constraint and any differences in the threat enumeration results can be attributed to boundary design choices rather than to specific implementation details. Furthermore, the model-level analysis allows the comparison to be conducted symmetrically, applying the same DFD notation, the same STRIDE

procedure, and the same attacker model to both variants, ensuring that the results are directly comparable [10], [11].

2.4 Related Work

2.4.1 Security in Microservice Architectures

Research on microservice security in the academia has grown considerably over the past decade, and several systematic literature reviews now provide a summarized view of what is and is not well understood. Berardi et al. conducted one of the largest of such reviews, analyzing 290 publications and classifying them across multiple security dimensions [6]. Their findings reveal that the majority of published work addresses communication-level security, specifically transport encryption and service authentication, while architectural security and access control receive comparatively less attention. Moreover, they find that only a small fraction of publications take a architectural perspective on microservice security, and that security is most commonly treated as an implementation detail rather than a design concern [6].

Hutasuhut et al., in a 2024 systematic review, confirm that access control remains the most critical security factor for microservice systems and that no generic security model currently exists for this architectural pattern [16]. Their review identifies a consistent gap between academic research and industry practice, and shows that while practitioners deal with access control problems frequently, the academic literature provides few generalizable frameworks for addressing them during the architectural stage [16]. Furthermore, de Almeida and Canedo find that the challenge of managing fine-grained, per-service authorization is consistently ranked among the top unsolved problems in microservice security, and that no single proposed mechanism adequately addresses the combination of consistency, scalability, and independence that the microservice pattern demands [24].

Leines-Vite et al. approach the problem by cataloging different security solutions used in microservice systems, mapping them to the STRIDE threat categories and the McCumber Cube dimensions of data state [23]. Their work confirms that solutions such as mTLS, JWT, OAuth 2.0, and RBAC are widely recommended, but that the correct selection and combination depends on where in the architecture the threat arises, which in turn depends on service boundaries and trust assumptions that exist between them [23]. Consequently, the solution catalog alone is insufficient without a corresponding architectural

model that specifies where each solution or standard must be applied.

2.4.2 DDD-Aligned Decomposition in Practice

The use of DDD bounded contexts as a microservice decomposition basis is well established in the industry and has been documented academically, among others, by Hippchen et al. [7]. Their work provides a detailed, process-oriented account of how DDD activities are applied in a real development project, demonstrating that the bounded context decomposition produces microservices with stable and well-defined interfaces, reduced coupling, and clear ownership of the domain logic. The study also demonstrates that DDD-aligned microservice teams experience fewer coordination problems, since each team works within a clearly delimited context with its own ubiquitous language and domain model [7].

However, the focus of Hippchen et al. and of the broader DDD-for-microservices literature is structural quality, namely cohesion, coupling, modularity, as well as the alignment between team and service boundaries [7]. The security implications of DDD decomposition are not examined. As such, the question of whether a bounded context, by concentrating domain logic and its rules within a single service boundary, also concentrates the authorization decisions relevant to that domain, and whether this concentration produces a more consistent authorization enforcement, is not addressed by the existing literature.

2.4.3 Zero Trust Architecture

Zero Trust Architecture is a network security model organized around the principle of never trusting and always verifying, in which no access subject is granted trust on the basis of its network location and every subject is treated as untrusted until it has been explicitly verified [19]. He et al. motivate this model by the structural weakness of perimeter-based protection, where components inside the internal network are trusted by default and a subject that has authenticated once retains that trust over a long period, allowing a malicious subject that passes authentication to continue attacking the internal network [19]. Under a Zero Trust Architecture, trust is instead established through identity authentication combined with continuous trust evaluation, permissions follow a least-privilege policy, and every request from an untrusted subject passes through a policy decision and enforcement point before any enterprise resource is reached [19].

Ghasemshirazi et al. survey the application domains, adoption challenges, and future opportunities of the zero trust paradigm, describing continuous verification, least-privilege access, and strict segmentation as its defining elements, where the network is divided into smaller isolated segments and each segment forms its own enforcement perimeter that limits the lateral movement of an attacker [29]. Furthermore, their survey identifies microservice and API security as an application domain in which zero trust principles govern the communication between services, so that only verified and authorized entities may invoke the interfaces a service exposes [29]. However, they also document substantial adoption obstacles, including organizational resistance to the required cultural shift, the technical complexity of integration with existing infrastructure, and the difficulty of balancing strict verification against usability [29].

The zero trust model frames the comparison conducted in this thesis, since the two architecture variants occupy opposite positions on the trust spectrum that this literature describes. The baseline variant deliberately instantiates the perimeter-trust approach, in which services consolidated within a shared boundary container accept requests from one another under implicit trust, which is exactly the assumption that zero trust rejects [19]. The DDD-aligned variant, in contrast, requires an independent grounded authorization decision at every inter-service delivery point, which corresponds structurally to the never trust, always verify principle applied at the granularity of bounded contexts [19], [29]. However, both surveys treat verification and segmentation primarily as network-level and infrastructure-level mechanisms, and neither addresses how the placement of segment boundaries should relate to domain responsibilities, so the connection between boundary alignment strategy and authorization enforcement consistency examined in this thesis remains unaddressed by the zero trust literature.

2.4.4 Research Gap

The reviews by Berardi et al. and Hutasuhut et al. both identify gaps in microservice security research, and while domain-driven approaches are mentioned, they appear in a small minority of publications, with comparatively little attention given to their security implications [6], [16]. As a result, the existing work on DDD-aligned microservice decomposition addresses mostly structural quality but not security [7]. Moreover, the existing work on microservice authorization identifies the consistency problem and categorizes mechanisms for addressing it, but does not connect the potential cause of

that problem, namely boundary design choices, to the security outcome in a systematic way [2], [3], [24].

Moreover, no published study has directly compared DDD-aligned and non-DDD-aligned microservice architectures precisely in terms of authorization enforcement consistency, using threat modeling as the evaluation method for such a comparison, while operationalizing authorization enforcement consistency as a measurable, model-derived property of a microservice architecture design. This thesis aims to address all three of these gaps, by constructing two contrasting architecture variants, applying STRIDE systematically to both, and deriving quantitative metrics from the resulting DFD models to compare authorization enforcement consistency as a direct function of the boundary alignment strategy.

Chapter 3

Method

This chapter describes the research methodology adopted to investigate whether DDD-aligned microservice boundary placement produces more consistent authorization enforcement than a baseline decomposition strategy. The chapter is structured as follows. Section 3.1 presents an overview of the research process and its sequential structure. Section 3.2 characterizes the underlying research paradigm. Section 3.3 defines the study design, including the controlled parameters, the independent variable, the attacker model, and the rationale for the design-level comparative approach. Section 3.4 addresses alternative methodologies that were considered and explains why each was rejected. Section 3.5 defines the four evaluation metrics used to compare the two architecture variants. Section 3.6 specifies the validity and reliability controls governing both the method and the data it produces. Finally, Section 3.7 briefly identifies the modeling and documentation tools employed.

3.1 Research Process Overview

The study is a controlled comparison, where two architecture variants, DDD-aligned and baseline, are built from the same e-commerce marketplace domain, the same four privileged operations, and the same attacker model, and they differ in whether their trust boundaries follow DDD bounded contexts or not. Both variants are modeled and analyzed in exactly the same way with DFDs and the STRIDE framework, so that the four evaluation metrics defined in Section 3.5 isolate the effect of that single boundary-placement choice and nothing else.

The research process that realizes this comparison consists of six sequential phases, namely domain scoping, variant design, DFD construction,

STRIDE enumeration, metric computation, and comparative analysis, whose dependency structure is shown in Figure 3.1. Each phase produces an artifact that is consumed by the subsequent phase, and no phase is begun before its predecessor is complete, which serves as a validity control, discussed further in Section 3.6.

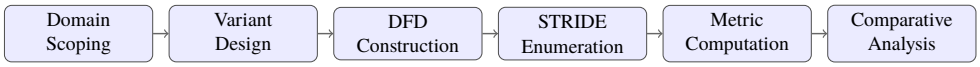


Figure 3.1: Overview of the research process.

In the domain scoping phase, the e-commerce marketplace domain is defined by identifying four functional areas, selecting four privileged operations that span those areas and defining the three actor roles operating within them, as documented in Section 4.1 of Chapter 4. This phase also fixes the attacker model, as defined in Section 3.3.3, and the authentication infrastructure assumptions that are held constant throughout the remainder of the study.

During the variant design phase, two architecture variants are constructed independently from the same domain scope. The DDD-aligned variant organizes service boundaries to coincide with bounded context boundaries, assigning each bounded context ownership of the domain logic, the data stores, and the authorization decisions that govern operations within it [4], [7]. On the other hand, the baseline variant derives its service boundaries from technical and workflow considerations rather than from domain ownership, producing a coarser decomposition, in which related operations are co-located within broader service units. The two variants share the same domain scope and the same set of privileged operations, with the boundary alignment strategy as the sole design variable distinguishing them.

In the DFD construction phase, a system-level DFD is produced for each variant using the four standard element types defined by the DFD meta-model, namely external entities, processes, data stores, and data flows, with explicit trust boundaries, in accordance with the modeling conventions defined in Section 3.3.4 [8], [9]. In addition, four per-operation authorization flow diagrams are derived for each variant, one for each privileged operation, making the authorization checkpoint structure of each operation explicit at the flow level.

The STRIDE enumeration phase applies the STRIDE-per-element procedure to each variant's system-level DFD and to each per-operation authorization flow diagram, restricted to the three in-scope STRIDE

categories defined in Section 3.3.5 [9], [11]. The enumeration is performed independently for each variant and only after both DFDs are finalized, so that threat findings in one variant cannot influence the design of the other.

In the metric computation phase, the four evaluation metrics defined in Section 3.5 are derived from the DFDs and the STRIDE enumeration results. Each metric value is computed independently for both variants from the same counting definitions, and every quantity is directly reproducible from the diagrams without requiring additional interpretive judgment.

Finally, in the comparative analysis phase, the metric values for the two variants are compared and interpreted in relation to the research question and the hypothesis. The analysis examines not only the numeric differences but also the structural properties of each variant's authorization enforcement configuration that produce those differences, as documented in Chapter 5.

3.2 Research Paradigm

The investigation is situated within a positivist research paradigm, in the sense that it assumes security properties of a software architecture are objective structural features that can be measured through a defined procedure applied to a formal model, and that the measured values can be compared across variants to evaluate a stated hypothesis [8], [9], [30]. The research approach is deductive and a working hypothesis is stated before any variant is designed [31]. The two variants are constructed under fixed and controlled assumptions, and the hypothesis is then assessed against the metric evidence derived from those models rather than developed inductively from the threat enumeration results.

The study combines qualitative and quantitative components. The qualitative component consists of the structured design and documentation of the two architecture variants, the application of DDD decomposition principles to derive the service boundaries of the DDD-aligned variant, and the application of STRIDE-per-element to identify the authorization-relevant threats in each [4], [9]. The quantitative component consists of deriving four model-level metrics from the DFDs and the threat enumeration, and comparing those metrics across the two variants to assess whether the DDD-aligned boundary strategy produces a measurably higher degree of authorization enforcement consistency [12], [13].

This paradigm is appropriate for the research question because the question is structural rather than empirical, focusing on whether a design-level choice produces a measurable architectural difference, rather than whether a deployed

system is more or less secure at runtime. The positivist assumption that the structural properties can be measured from a formal model is directly supported by the established use of DFDs and STRIDE-per-element as reproducible, structure-driven analysis tools in both academic and industrial threat modeling practice [8], [9], [10].

3.3 Comparative Study Design

The study is structured as a controlled, design-level comparative analysis in which a single independent variable, the boundary alignment strategy, is varied across two architecture variants while all other parameters are held constant [32]. This structure ensures that any measured difference in authorization enforcement consistency can be attributed to the boundary strategy rather than to incidental differences in scope, modeling precision, or assumed infrastructure. The comparative design is appropriate here because the research question asks whether a structural design choice produces a measurable architectural difference, and a comparison of two fully specified variants under controlled conditions is the most direct method for generating that evidence [8], [9].

3.3.1 Controlled Parameters

The parameters held constant across both variants are the domain scope, the set of privileged operations, the attacker model, the authentication infrastructure assumptions, the DFD modeling conventions, and the STRIDE enumeration procedure. The domain scope is a single e-commerce marketplace consisting of four functional areas, namely identity and access management, product cataloging, order processing, and financial settlement, together with four privileged operations that span those areas and the three actor roles operating within them, as specified in Table 4.1 of Section 4.1. The attacker model is fixed as an authenticated low-privilege user holding a valid Buyer role identity token and attempting those four privileged operations. This model applies without modification to both variants, as detailed in Section 3.3.3. The authentication and identity infrastructure is assumed equivalent in both variants, specifically a shared identity provider issuing cryptographically signed tokens, so that any observed differences in authorization enforcement cannot be attributed to differences in the authentication layer. The DFD modeling conventions, including the element types used, the granularity at which services are represented, and the rules for placing trust boundaries,

are applied consistently to both variants in accordance with Section 3.3.4. The STRIDE-per-element procedure is applied to both DFDs using the same enumeration rules and the same scope restriction, as defined in Section 3.3.5 [8], [9].

The set of services and their data stores is held constant across both variants, and the boundary alignment strategy varies only how those services are grouped into trust boundaries. The DDD-aligned variant draws its boundaries by domain ownership, while the baseline draws them by technical and workflow considerations. The resulting difference in the number of trust-boundary containers, and therefore in the number of boundary-crossing flows, is a structural expression of that strategy rather than an independent variable requiring separate control.

3.3.2 Independent and Dependent Variables

The independent variable is the boundary alignment strategy, where the DDD-aligned variant applies domain-driven bounded context boundaries as explicit trust boundaries, while the baseline variant does not apply domain-aligned boundary placement. The dependent variable is authorization enforcement consistency, operationalized through the four evaluation metrics defined in Section 3.5. All other design dimensions are held constant as specified in Section 3.3.1, so that the metric values measured for each variant can be attributed to the boundary alignment strategy alone.

3.3.3 Attacker Model

The attacker is modeled as an authenticated low-privilege user holding a valid Buyer role identity token. This choice reflects the threat scenario of greatest relevance to the research question, namely a principal who has legitimately authenticated to the system and is attempting to exceed the authorization boundary associated with that identity by exploiting gaps in the authorization enforcement architecture, rather than weaknesses in the authentication infrastructure itself. Accordingly, threats that presuppose the absence of a valid identity token, such as credential enumeration, session fixation, or authentication bypass at the identity provider, are outside the scope of this analysis [2].

The attacker is assumed to possess a valid cryptographically signed identity token, to have network-level access to the API Gateway, and to have no elevated network access to the internal service mesh or any individual

backend service. The attacker's four goals correspond directly to the four privileged operations defined in Section 4.1, namely publishing a product listing without holding the Seller role, placing an order and triggering a payment capture without the payment step performing an independent role verification, obtaining a refund against an order not owned by the attacker's identity, and overriding a disputed order and its associated settlement record without holding the Administrator role. These goals are applied symmetrically to both variants, so that the set of authorization challenges each variant is evaluated against is identical.

The attacker model is intentionally narrow. An unauthenticated external attacker or a privileged insider would introduce threat categories that are not the focus of this study, since both variants apply the same authentication infrastructure and neither attempts to address infrastructure-level compromise. A compromised internal service would introduce lateral movement paths whose structure does differ between the variants, however, this scenario presupposes an attacker who has already achieved service-level code execution, which would already imply a breach of the trust boundary assumptions under evaluation. Restricting the model to an authenticated low-privilege user therefore isolates the architectural effect of boundary alignment on authorization enforcement from the effects of authentication strength, transport security, and infrastructure-level trust, which are held constant across both variants [3].

3.3.4 DFD Modeling Conventions

The system-level DFD for each variant is constructed using the four standard element types defined by the DFD meta-model, namely external entities, processes, data stores, and data flows, with trust boundaries represented as named boundary containers that group elements operating at the same privilege level [8], [9]. This convention is applied without modification to both variants, so that any structural difference observable in the resulting diagrams can be attributed to the boundary alignment strategy rather than to differences in modeling notation or element granularity.

Whether a data flow constitutes a trust-boundary crossing is determined directly from the DFD topology, specifically, any flow whose source process and destination process reside in different boundary containers constitutes a trust-boundary crossing and is therefore subject to the full STRIDE-per-element enumeration scoped to the categories defined in Section 3.3.5 [33]. Conversely, any flow whose source and destination reside within the same

boundary container does not cross a trust boundary and is consequently excluded from that enumeration [8], [9]. Consequently, the placement of trust boundary containers, and specifically the decision of whether two services share a container, directly determines the scope of the authorization threat enumeration and is therefore itself a measured structural outcome of the boundary alignment strategy under investigation.

The granularity at which services are represented is held constant across both variants in the sense that each independently deployable service is represented as a single process node, and each service-owned data store is represented as a single data store node co-located with its owning service in the same boundary container. In the DDD-aligned variant, one boundary container is defined per bounded context, so that no two backend services share a container and every inter-service flow constitutes a trust-boundary crossing. In the baseline variant, the boundary container assignment follows the technical grouping strategy of that variant, and the sharing of containers between services is a structural consequence of that strategy rather than a modeling choice [2], [3]. The per-variant element lists and trust boundary assignments are documented in Sections 4.2.2 and 4.3.2 respectively.

3.3.5 STRIDE Scope and Category Justification

The STRIDE analysis in this study is restricted to three of the six threat categories, namely Elevation of Privilege, Spoofing, and Tampering, on the basis of their direct relevance to the defined attacker model and the authorization enforcement properties being measured [3], [9]. Elevation of Privilege is the primary category, since all four attacker goals are instances of an authenticated low-privilege principal attempting to exceed the authorization boundary associated with its role, which is the defining characteristic of this threat category [9]. Spoofing is included at inter-service trust-boundary crossings because a service that accepts a spoofed identity claim applies authorization based on incorrect identity context, which is structurally equivalent to an authorization bypass at that boundary, and therefore directly contributes to the authorization enforcement gap count [34], [2], [23]. Tampering is included for data flows carrying authorization tokens or propagated ownership context, because modification of those flows can elevate the effective privilege of the attacker without a direct authorization failure being recorded at any individual service [2], [23], [35].

The remaining three STRIDE categories are excluded on the basis that they fall outside the defined attacker's goals and do not contribute evidence

relevant to the research question. Information Disclosure threats concern the confidentiality of data rather than the correctness of authorization decisions, and while they may be present in the DFDs, their enumeration would not affect any of the four evaluation metrics defined in Section 3.5. Repudiation threats concern the absence of non-repudiation controls rather than access control failures, and similarly, do not bear on authorization enforcement consistency. Denial of Service threats concern availability rather than authorization, and their presence or absence is not a function of the boundary alignment strategy under investigation. Implementation-level vulnerability classes such as SQL injection, XSS, cryptographic implementation flaws, and third-party library CVEs are similarly out of scope, since they are not representable at the DFD level and would not differentiate the two variants under the modeling conventions defined in Section 3.3.4. The scope of the analysis is therefore the structural-design subset of authorization-relevant threats representable at the DFD level, and the restricted scope is not an approximation but a deliberate narrowing that preserves completeness with respect to the research question while avoiding the enumeration of threats that would not differentiate the two variants [9].

3.3.6 Design-Level Comparison Rationale

The design-level controlled comparison is the appropriate method for this research question because the question concerns the structural properties of an architecture variant, specifically whether the placement of trust boundaries relative to domain responsibilities affects the degree to which authorization enforcement is grounded in domain-owned state, rather than the runtime behavior of any deployed system. Sion et al. establish formally that DFDs capture the security-relevant structure of an architecture at a level that is both accessible and sufficient for systematic threat identification without requiring a running implementation [8]. Furthermore, the STRIDE-per-element procedure, as described by Shostack, is specifically designed to produce reproducible threat enumerations whose completeness is bounded by the completeness of the DFD rather than by the analyst's prior knowledge, making it well-suited to a controlled comparison in which the two variants must be analyzed under identical conditions [9]. Consequently, a design-level comparison provides a controlled and symmetrical basis for attributing metric differences to the boundary alignment strategy, rather than to implementation choices or runtime conditions that are not represented in the model.

3.4 Alternative Methodologies

Two alternative methodological approaches were considered before the design-level controlled comparison was selected, and both were rejected for reasons specific to the nature of the research question.

3.4.1 Runtime and Implementation-Based Testing

A runtime-based approach, for example deploying both variants as working systems and subjecting them to penetration testing or automated vulnerability scanning, would measure the security behavior of a specific implementation rather than the structural property of the design. Any findings would reflect implementation-specific decisions, library versions, and environmental conditions that are not controlled by the boundary alignment strategy under investigation [8], [36]. Furthermore, a runtime approach would require a complete, deployable implementation of each variant, which falls outside the feasible scope of this thesis and would introduce uncontrolled implementation variables that compromise the comparability of the two variants. The design-level DFD and STRIDE approach avoids both problems by operating directly on the architecture model rather than on a specific implementation of it.

3.4.2 Systematic Literature Review or Expert Survey

A systematic literature review or expert survey would produce aggregated scholarly opinion or practitioner judgment rather than a structural comparison of specific architecture variants. Chapter 2 already documents that no studies were identified that directly compare DDD-aligned and non-DDD-aligned microservice architectures in terms of authorization enforcement consistency [6], [16]. A literature review would confirm this gap but could not fill it, as it would not produce the structured comparative evidence that the research question requires. Similarly, an expert survey could not operationalize authorization enforcement consistency as a measurable, model-derived property of specific architecture designs, nor control the comparison with sufficient precision to confidently attribute any observed differences to boundary alignment alone.

3.4.3 Solution-Aware DFDs

Solution-aware DFD notation, as proposed by Sion et al., enriches the standard DFD element set with security solution annotations that make the mitigations

already present in an architecture visible within the model [8]. However, this enrichment is deliberately not applied in this thesis, and the same decision is applied symmetrically to both variants. The central measurement objective is to determine whether the structural placement of trust boundaries produces a difference in the authorization enforcement consistency metrics, and annotating mitigations would conflate the structural property being evaluated with the visualization of its remediation. The two variants would therefore no longer differ only in boundary alignment strategy but also in the density and distribution of solution annotations, weakening the traceability between boundary design choices and the metric outcomes derived from the STRIDE enumeration [8], [9].

3.5 Evaluation Framework

Authorization enforcement consistency is operationalized through four model-derived metrics, each defined precisely enough that its value for a given variant can be computed by inspection of the variant's DFD and per-operation authorization flow diagrams without requiring additional interpretive judgment. The four metrics address distinct but complementary structural dimensions of authorization enforcement, namely the coverage and quality of enforcement checkpoints at trust-boundary crossings, the rate of implicit-trust flows, the degree of co-location between authorization responsibility and domain responsibility, and the structural depth of the verification paths that grounded checkpoints rely upon. The metrics are presented in Table 3.1 and defined in full in the subsections that follow.

Metric	Definition	Computed from	Direction
M1	Ratio of grounded PEP instances to total inter-service delivery points participating in privileged operations (including delivery points with no PEP), regardless of whether the exchange crosses a drawn trust boundary	DFD inter-service request flow list and per-operation flow diagrams	Higher
M2	Ratio of implicit-trust flows to total privileged-operation request flows, including intra-container flows not crossing a drawn trust boundary	DFD privileged-operation request flow list and per-operation flow diagrams	Lower
M3	Mean per-operation fragmentation score, where each score is $1/k$ and k is the number of boundary containers hosting a grounded PEP instance contributing to that operation	DFD boundary container assignments and per-operation flow diagrams	Lower*
M4	Mean number of trust-boundary crossings each grounded PEP traverses on the round-trip verification sub-path to the authoritative data source, excluding the incoming privileged request crossing	DFD crossing list and per-operation flow diagrams	Lower*

M3 and M4 must each be interpreted jointly with M1.

Table 3.1: Evaluation metric definitions.

The three in-scope STRIDE categories feed the four metrics through the structural features they mark on each flow. Every privileged-operation request that reaches a receiving service carries an Elevation of Privilege finding, and M1 and M2 record opposite resolutions of that finding, with M1 counting the inter-service delivery points at which a grounded PEP resolves it against state the receiver owns and M2 counting the flows at which the receiver admits the request under implicit trust and leaves it unresolved. M3 then records where the grounded checkpoints that resolve those findings sit, counting the boundary containers across which they fall for each operation. The Spoofing and Tampering findings enumerated on the role-verification and ownership-confirmation flows mark the channels a grounded PEP must cross to reach authoritative state, and M4 counts those trust-boundary crossings.

3.5.1 M1: Grounded Enforcement Coverage

M1 measures the joint coverage and quality of authorization enforcement at the inter-service delivery points that participate in the four privileged operations. A delivery point is any receiving service that admits a privileged-operation request from another service, regardless of whether the exchange crosses a drawn trust boundary. Verification queries issued between services to ground a downstream authorization decision are not delivery points and are captured by M4 instead. For each such crossing, a PEP instance is considered grounded if it verifies authorization using state owned by the receiving service rather than relying solely on identity claims propagated from the upstream caller [2], [3]. A grounded PEP is therefore distinguished from a PEP that mechanically accepts a forwarded token without consulting domain state that belongs to the receiving service.

M1 is the ratio of grounded PEP instances to the total count of inter-service delivery points involved in privileged operations, including delivery points where no PEP is present. Including intra-container delivery points in the denominator prevents a coarser decomposition from achieving an artificially high M1 by absorbing privileged-operation requests into shared containers rather than by enforcing authorization independently at each receiving service. This makes M1 a joint measure of both enforcement coverage and enforcement quality, since a delivery point with no grounded PEP contributes zero to the numerator while still incrementing the denominator.

A low value of M1 indicates a structural enforcement gap of the kind that NIST SP 800-204 identifies as a per-service authorization deficit that the API gateway alone cannot remedy, and that Jayalath et al. classify under missing permission checks and improper propagation of identity context [3], [5]. M1 is computed from the DFD inter-service request flow list and the per-operation authorization flow diagrams, as described in Section 3.3.4.

3.5.2 M2: Normalized Implicit-Trust Rate

An implicit-trust flow is any inter-service data flow that carries a request involving a privileged operation to a receiving service that performs no independent authorization check on that request, regardless of whether the flow crosses a drawn trust boundary. M2 is the ratio of such flows to the total number of inter-service privileged-operation request flows across the variant.

The rate formulation rather than a raw count is used because the two architecture variants may produce structurally different numbers of inter-service flows by construction, depending on how services are grouped into

boundary containers. A variant with finer decomposition will produce more inter-service flows overall, giving more opportunities for implicit-trust flows to appear and making raw counts systematically biased toward architectures with fewer service boundaries. The normalized rate removes this structural bias and makes variants with different decomposition granularity directly comparable [2].

The denominator includes intra-container request flows as well as cross-boundary request flows, because a service that accepts a privileged request from a co-located service without performing an independent authorization check exhibits the same structural vulnerability as one that accepts such a request across a drawn trust boundary. Restricting the denominator to trust-boundary crossings alone would allow a coarser decomposition strategy to achieve an artificially low M2 score by absorbing implicit-trust flows into shared containers rather than eliminating them, which would introduce a systematic measurement bias in favor of less granular architectures.

A lower value of M2 indicates a lower degree of dependence on upstream services having already verified authorization on behalf of the downstream receiver. This property is directly connected to the lateral movement vulnerability that arises when a compromised or misconfigured upstream service forwards unauthorized requests that are accepted by downstream services without an independent authorization check [2], [5]. M2 is derived by direct inspection of the DFD privileged-operation request flow list and the per-operation authorization flow diagrams, as specified in Section 3.3.5.

3.5.3 M3: Authorization Fragmentation Score

M3 measures the degree to which the authorization decisions governing each privileged operation are concentrated within a single boundary container rather than distributed across multiple containers. For each of the four privileged operations, a per-operation fragmentation score is computed as $1/k$, where k is the number of distinct boundary containers that host a grounded PEP instance contributing to that operation. A container whose service receives a privileged request and admits it under implicit trust does not host a grounded PEP and is therefore not counted in k, since the metric is intended to measure how authorization decisions grounded in locally owned state are distributed across the architecture, not how many containers passively participate in the request chain. An operation whose authorization decisions all reside within one container receives a score of 1.0, and an operation whose decisions are distributed across k containers receives a score of $1/k$. M3 is the arithmetic

mean of the four per-operation scores.

The $1/k$ formulation produces a continuous score rather than a binary fully-contained or partially-contained classification, which is necessary for operations that span more than two containers, since a binary classification would treat a two-container split and a four-container split as equivalent. The score attains its ceiling of 1.0 when every grounded PEP instance contributing to an operation resides in a single boundary container, and decreases toward zero as those instances are distributed across additional containers. M3 is not interpretable on its own, because the same score can arise from structurally opposite configurations. An operation whose receiver admits the privileged request under implicit trust contributes no grounded PEP instance and is therefore counted as $k = 1$ by default, indistinguishable in M3 alone from an operation whose authorization is genuinely concentrated within one container. The metric is consequently read jointly with M1, which determines whether the containers counted in k host real grounded enforcement decisions or are placeholders produced by the counting rule. The directional prediction for M3 and the conditions under which it is treated as inconclusive are stated in the assessment paragraph at the end of this section [6], [16]. M3 is computed from the DFD boundary container assignments and the per-operation authorization flow diagrams, as specified in Section 3.3.4.

3.5.4 M4: Cross-boundary Verification Depth

M4 measures the structural resilience of the grounded PEP instances that M1 counts by quantifying how many additional trust-boundary crossings each grounded PEP must traverse to complete its authorization verification. For a grounded PEP hosted by service S, the verification depth is the count of distinct trust-boundary crossings on the verification sub-path, defined as the path from S to the authoritative data source and back, not counting the crossing that delivered the original privileged request to S.

A grounded PEP that verifies authorization using only data stored locally within its own service boundary has a verification depth of zero, because no outbound trust-boundary crossing is required. A PEP that must call one external service and receive a response traverses two crossings, one outbound and one inbound, and therefore has a depth of two. When a grounded PEP issues several independent verification sub-paths against distinct authoritative sources, for example one sub-path to the identity store to verify role and a second to a domain data store to confirm resource ownership, its verification depth is the sum of the depths of all such sub-paths. M4 is the arithmetic mean

of the verification depths across all grounded PEP instances in the variant. A verification flow that exists in the DFD but is not issued by a grounded PEP contributes zero to M4, because the metric is defined as a mean over the set $\mathcal{P}$ of grounded PEP instances only.

A lower value of M4 indicates that grounded authorization decisions are structurally resilient, in that they depend on fewer inter-service round trips and are therefore less sensitive to cascading failures, network partitions, or the compromise of intermediate services on the verification path [3], [2]. M4 must be interpreted jointly with M1, because a low M4 achieved by removing grounded PEP instances rather than by co-locating authorization state with the enforcing service would represent a structural regression rather than an improvement. The analytically strongest outcome is a variant that achieves M1 =1.0 combined with a low M4, indicating both complete enforcement coverage and structurally resilient verification paths [5]. A higher M4 achieved alongside M1 = 1.0 is acceptable only when two structural properties hold: each verification sub-path terminates at a single authoritative data store rather than propagating through intermediate services, and the number of distinct authoritative sources any single PEP instance must consult is bounded by the logical complexity of the operation's authorization requirements rather than by the system's service count. When both properties hold, M4 grows in proportion to what the authorization logic demands and not arbitrarily. When either fails, a rising M4 signals compounding verification-chain complexity and constitutes a structural liability regardless of M1. The analysis in Chapter 5 confirms whether these properties hold for the operations examined in this study. M4 is computed from the DFD trust-boundary crossing list and the per-operation authorization flow diagrams.

3.5.5 Flow Classification Rules

The four metric definitions above operate on three semantically distinct classes of DFD flow. Table 3.2 summarises which class contributes to which metric, in variant-agnostic terms, and serves as the operational counting rule applied uniformly to both architecture variants. The per-operation instantiations of this classification for the two variants of this study are listed alongside the metric-input tables of Sections 5.2.1–5.2.4.

Flow class	M1	M2	M3	M4
Privileged-operation request (inter-service)	Counted in denominator, and in numerator when receiver has a grounded PEP	Counted in denominator, and in numerator when receiver has no grounded PEP	Receiver's container counted in k when it hosts a grounded PEP	—
Verification flow (inter-service)	—	—	—	Each crossing adds one unit to the issuing PEP's depth
Non-inter-service flow	—	—	—	—

Table 3.2: Flow classification under the evaluation framework. A dash indicates the flow class does not contribute to that metric.

The hypothesis is assessed against four directional predictions, each of which the DDD-aligned variant must satisfy relative to the baseline variant. The first prediction is that M1 is higher in the DDD-aligned variant. The second is that M2 is lower in the DDD-aligned variant. The third prediction concerns M3 and is conditional on the M1 outcome, since M3 is not interpretable in isolation, as established in Section 3.5.3. When M1 attains its ceiling value of 1.0 in the DDD-aligned variant and the baseline variant records a strictly lower M1, the predicted direction for M3 is lower in the DDD-aligned variant, reflecting distributed grounded enforcement across multiple boundary containers rather than concentration through implicit trust. When M1 = 1.0 does not hold in the DDD-aligned variant, no directional prediction is made for M3 and that metric is treated as inconclusive for assessment purposes. The fourth prediction is that M4 is equal to or lower in the DDD-aligned variant than in the baseline variant.

A result in which all four directional predictions hold constitutes full support for the hypothesis. A result in which three of the four predictions hold, with the diverging metric admitting a structural explanation that does not reverse the hypothesis itself, constitutes partial support. A result in which the baseline variant matches or outperforms the DDD-aligned variant on two or more predictions constitutes evidence against the hypothesis. The structural conditions producing any diverging metric require explicit explanation in the analysis, and the verdict reached must be derivable mechanically from the per-metric directional outcomes against the rule stated here, as carried out in

Chapter 5.

3.6 Assessing Reliability and Validity

The validity and reliability of this study are assessed along two dimensions [37], [38]. Construct validity concerns whether M1 through M4 measure authorization enforcement consistency rather than some other structural property of the two variants, and whether the comparison between the variants is governed by modeling rules that prevent the researcher from favoring one variant through asymmetric design choices. Reproducibility concerns whether another analyst applying the same procedure to the same DFD would obtain the same threat enumeration and metric values, and what steps were taken to ensure that the DFD models are sufficiently complete to support that enumeration.

3.6.1 Construct validity

The four metrics are grounded in structural features of the DFD that the authorization literature identifies as causes or indicators of enforcement inconsistency, not in implementation artifacts such as source code, configuration files, or runtime traces [8], [9], [37]. M1 corresponds to the per-service, fine-grained authorization that OWASP and NIST SP 800-204 prescribe as distinct from gateway-level filtering [2], [3]. M2 corresponds to the lateral movement vulnerability that arises when a downstream service admits a forwarded request without an independent check, classified by Jayalath et al. under missing permission checks and improper propagation of identity context [5]. M3 corresponds to the co-location of domain and authorization responsibility whose absence Berardi et al. and Hutasuhut et al. identify as a structural gap [6], [16], and finally M4 corresponds to the structural resilience of grounded PEP instances, measured by the number of inter-service round trips each must complete before admitting a privileged operation [3]. The metric definitions in Section 3.5 make the mapping from structural property to metric value precise enough that each value can be computed by inspection of the DFD and per-operation flow diagrams without additional interpretive judgment [38].

Construct validity also requires that the comparison be fair, meaning that neither variant benefits from more generous modeling assumptions than the other [37]. Three controls address this. First, both variants are constructed from the same fixed domain scope, the same set of privileged operations,

the same element-type definitions, and the same trust-boundary placement rules specified in Section 3.3.4. The only input that differs between them is the boundary alignment strategy that defines each variant. Second, the boundary configuration of the baseline variant is not an ad hoc choice made to maximize the metric delta but an instance of the documented perimeter-trust pattern that NIST SP 800-204 and OWASP identify as a structural source of authorization enforcement gaps in operational microservice deployments [2], [3]. Its selection and its limitations relative to other non-DDD-aligned configurations are discussed explicitly in Sections 4.3.1 and 6.2. Third, the element granularity rule stated in Section 3.3.4, namely one process node per independently deployable service, one data store node per service-owned data store, co-located with its owning service, applies without modification to both variants, so that differences in the number of trust-boundary crossings visible in the two DFDs are a structural consequence of the boundary alignment strategy rather than of inconsistent modeling precision. In addition, both DFD models were finalized in full before any STRIDE enumeration or metric computation began, so that intermediate metric observations could not influence boundary placement in either variant.

3.6.2 Reproducibility

The STRIDE-per-element procedure determines the threat enumeration directly from the element types and boundary crossings present in the DFD, eliminating dependence on the analyst's prior knowledge of specific attack patterns [9], [11]. Given the element-type-to-category mapping of Khan et al., the attacker model and scope restriction of Sections 3.3.3 and 3.3.5, and the counting definitions of Section 3.5, the enumeration and the resulting metric values are reproducible by any analyst working from the same DFD without additional interpretive judgment [38].

This determinism is a property of the counting procedure given a fixed input, and it is distinct from the prior question of whether the DFD captures every authorization-relevant flow in the architecture. The procedure reproduces faithfully whatever a complete DFD expresses and reproduces equally faithfully whatever an incomplete one omits, which makes model completeness a precondition of valid metric values rather than something the procedure itself can verify.

Two measures were taken to achieve reasonable model completeness within the defined scope. First, the STRIDE-per-element procedure was applied to every element of each DFD, restricted to the three threat categories

directly relevant to the attacker model, Elevation of Privilege, Spoofing, and Tampering, applied at trust-boundary crossings and flows that carry or consume authorization context for the four selected privileged operations, as defined in Section 3.3.5 [27]. The completeness claim is therefore scoped, not that every possible threat is enumerated, but that every Elevation of Privilege, Spoofing, and Tampering threat at a trust-boundary crossing or privileged-operation flow present in the DFD is enumerated. Second, both DFDs were reviewed against the four attacker goals specified in Section 3.3.3 to confirm that every inter-service flow the attacker would traverse to reach a privileged operation is represented in the diagram. Flows that satisfy this check for both variants are documented in the per-operation flow classification tables of Sections 5.2.1-5.2.4.

The two variants were constructed sequentially and independently as described in Section 3.1, with the STRIDE analysis applied to both DFDs only after both designs were finalized, so that threat findings in one variant could not influence the placement of enforcement checkpoints in the other. However, the entire procedure was executed by a single researcher, which is a limitation on both reproducibility and model completeness, as no independent reviewer confirmed that the DFD models capture every authorization-relevant interaction, and no inter-rater check was performed on the metric counts. These limitations are addressed in Section 6.2.

3.7 Software and Modeling Tools

The DFD models for both variants are constructed using OWASP Threat Dragon, a threat modeling tool that supports the DFD element types and trust boundary containers required by the modeling conventions defined in Section 3.3.4. No automated threat modeling tools are used. The STRIDE-per-element enumeration is performed manually from the DFDs, in accordance with the procedure described by Shostack and extended by Khan et al. [9], [11]. This manual approach is appropriate for the scope of the study, since the DFD models are sufficiently small and the enumeration procedure is sufficiently structured that automated tooling would not improve the reliability of the results relative to a careful manual application.

3.8 Summary

This chapter has fixed everything that the comparison in Chapter 5 depends on, as well as described the controlled parameters, the attacker model, and the modeling conventions that keep the two variants comparable. Additionally, it defined the four metrics M1 to M4 together with the directional predictions and the rule that separates full support, partial support, and evidence against the hypothesis. Chapter 4 now applies this methodology to construct each variant's DFD and STRIDE enumeration, and Chapter 5 then reads the metric values from those models and applies the assessment rule. The metric definitions and pre-committed assessment criterion are therefore carried forward, since the verdict in Chapter 5 follows mechanically from them.

Chapter 4

Threat Modeling

This chapter applies the methodology defined in Chapter 3 to the specific domain and architecture variants under investigation. Section 4.1 defines the e-commerce marketplace domain scope, specifying the four functional areas, the three actor roles, and the four privileged operations that are applied symmetrically to both architecture variants. Section 4.2 documents the DDD-aligned architecture variant, presenting its service decomposition, its system-level DFD with explicitly placed trust boundaries, and the per-operation authorization flow diagrams derived from that DFD. Section 4.3 presents the corresponding artifacts for the baseline variant. The outputs of both sections constitute the primary evidence from which the evaluation metrics in Chapter 5 are derived and against which the research hypothesis is assessed [9].

4.1 Domain and Privileged Operations

The e-commerce marketplace domain was selected because it encompasses four functionally distinct areas, namely identity and access management, product cataloging, order processing, and financial settlement, each of which carries well-defined ownership responsibilities and authorization rules, and because the interactions among those areas produce cross-area authorization dependencies of exactly the kind that the investigation is designed to expose [2]. The three actor roles operating across these areas are Buyer, Seller, and Administrator, whose privilege levels and resource ownership relationships vary in ways that generate structurally distinct authorization challenges when service boundaries are drawn along different lines. The controlled complexity of this domain, consisting of four functional areas and four privileged operations, is sufficient to yield analytically distinct threat profiles between the

two variants while remaining feasible for a design-level comparative analysis that does not require a running implementation.

Operation & Role	Areas Involved	Description
OP-1 [Seller] Listing Publication	Product Cataloging	Seller submits a new product listing for publication in the catalog.
OP-2 [Buyer] Order and Payment	Order Processing Financial Settlement	Buyer places an order and triggers a payment capture across the order and financial settlement functional areas.
OP-3 [Buyer] Return and Refund	Order Processing Financial Settlement	Buyer requests a return on existing order. The financial settlement area must authorize a refund against an ownership record held in the order functional area, producing a cross-area ownership dependency.
OP-4 [Administrator] Dispute Override	Order Processing Financial Settlement	Administrator overrides a disputed order and its associated settlement record. The operation affects both functional areas and is irreversible once committed.

Table 4.1: Privileged operation selection.

The four operations provide coverage of all three actor roles and of the two structurally distinct authorization challenge types identified in the microservice security literature, namely role-sufficiency checks, in which the correct role is the necessary and sufficient condition for authorization, and ownership-verification checks, in which role alone is insufficient and the enforcing area must additionally confirm a resource ownership relationship [5], [39], [40]. Operations OP-1, OP-2, and OP-4 are role-sufficiency cases, with OP-2 being the cross-area instance of this type, requiring both the order processing and the financial settlement areas to independently verify the Buyer role rather than accept the role assertion forwarded across the area boundary. Operation OP-3 is the sole ownership-verification case, producing the sharpest cross-area dependency, since the financial settlement area must validate the ownership of the return request against the authoritative record in the order functional area.

The two variants introduced in the following sections share the same four independently deployable services and the same four data stores; the single structural parameter that distinguishes them is trust-boundary placement, which in turn determines where grounded PEPs are structurally required. Table 4.2 previews both variants before the detailed models that follow.

Variant	Services	Trust Boundaries	Data Stores	PEP Locations
DDD-aligned	Identity, Catalog, Order, Payment (4 services)	One per backend service (6 containers)	Each co-located inside its service's container (4 stores)	Grounded PEP at each receiving backend service
Baseline	Identity, Catalog, Order, Payment (4 services)	Business services share one container (4 containers)	Identity separate, Catalog, Order, Payment co-located (4 stores)	Grounded PEP at gateway-facing service only

Table 4.2: Summary of the architecture variants.

Flow identifiers are shared across the two variants and table 4.3 maps each of them to their purpose. Flows F-12 and F-13 occur only in the DDD-aligned variant, where the Payment Context hosts its own grounded PEP.

Flow	Operations	Purpose
F-01	OP-2, OP-3	Buyer order and return request entry to the API Gateway
F-02	OP-1	Seller listing publication request to the API Gateway
F-03	OP-4	Administrator settlement override request to the API Gateway
F-04	OP-1	API Gateway to Catalog, routed listing publication request
F-05	OP-2	API Gateway to Order, routed order placement request
F-06	OP-3	API Gateway to Order, routed return request
F-07	OP-4	API Gateway to Order, routed settlement override request
F-08	OP-1	Catalog to Identity, role verification call
F-09	OP-1	Identity to Catalog, role confirmation response
F-10	OP-2–OP-4	Order to Identity, role verification call
F-11	OP-2–OP-4	Identity to Order, role confirmation response
F-12	OP-2–OP-4	Payment to Identity, role verification call (DDD)
F-13	OP-2–OP-4	Identity to Payment, role confirmation response (DDD)
F-14-A	OP-3	Order to Payment, refund authorization request
F-14-B	OP-3	Order to Payment, ownership confirmation response
F-14-C	OP-2	Order to Payment, payment capture request
F-14-D	OP-4	Order to Payment, settlement override request
F-15	OP-3	Payment to Order, ownership confirmation request
F-16	OP-1	Catalog to Catalog Data Store, listing read/write
F-17	OP-2–OP-4	Order to Order Data Store, order write or ownership read
F-18	OP-2–OP-4	Payment to Payment Data Store, settlement read/write
F-19	OP-1–OP-4	Identity to Identity Data Store, role record read

Table 4.3: Summary of DFD flow identifiers and operation mapping.

4.2 The DDD-Aligned Variant

The DDD-aligned variant decomposes the e-commerce marketplace domain into four independently deployable services, each corresponding directly to one of the four functional areas identified in Section 4.1. Each service is assigned exclusive ownership of the domain logic, the data stores, and the authorization decisions that govern operations within its bounded context [4], [7]. The decomposition follows the process described by Hippchen et al., in which the domain model is partitioned into bounded contexts based on organizational domain ownership, and each context is then modeled as a single deployable service that exposes well-defined interfaces for communication with neighboring contexts [7]. Consequently, the four bounded contexts in this variant are the Identity Context, the Catalog Context, the Order Context, and the Payment Context, each of which owns its own data store and is the sole authority for authorization decisions involving the resources it manages.

4.2.1 Service Decomposition

The Identity Context owns the Identity Data Store and is the sole authority for issuing and validating role assertions across all other contexts. The Catalog Context owns the Catalog Data Store and is responsible for managing product listing records, including the authorization constraints governing listing publication. The Order Context owns the Order Data Store and is responsible for managing order records and the ownership relationships associated with them. The Payment Context owns the Payment Data Store and is responsible for managing settlement and refund records. No two contexts share a data store, and no context may read from or write to a data store owned by another context [4], [7].

Each bounded context enforces its own authorization decisions by hosting a co-located PEP instance that performs an independent cross-boundary role verification call to the Identity Context before admitting any privileged operation to domain logic [2], [3]. This arrangement ensures that no context relies on a role assertion forwarded by an upstream context, so the authorization outcome at each context is grounded in the authoritative record held by the Identity Data Store rather than in a claim propagated across the boundary from a prior context.

4.2.2 System-Level DFD

The system-level DFD for the DDD-aligned variant is constructed by applying the modeling conventions defined in Section 3.3.4 to the bounded context decomposition described in Section 4.2.1. Six trust boundary containers are defined for this variant, each corresponding to a distinct domain in the architecture. TB-0 contains the external user population, represented as the Buyer, Seller, and Administrator external entities. TB-1 contains the API Gateway process, which performs coarse-grained authentication verification and routes incoming requests to the appropriate bounded context. TB-2 through TB-5 each contain a single bounded context service together with its co-located data store, specifically the Identity Context service and its Identity Data Store in TB-2, the Catalog Context service and its Catalog Data Store in TB-3, the Order Context service and its Order Data Store in TB-4, and finally the Payment Context service and its Payment Data Store in TB-5. Consequently, no two backend services share a boundary container, which means that every inter-service communication flow in this variant constitutes a trust-boundary crossing and is subject to the full STRIDE-per-element enumeration scoped to the categories defined in Section 3.3.5 [8], [9].

The only intra-container flows in this variant are those in which a bounded context service reads from or writes to its own co-located data store, since each data store resides within the same boundary container as the service that owns it. For inter-service flows, every call from the API Gateway to a bounded context and every cross-context verification call between bounded contexts constitutes a trust-boundary crossing, since no two backend services share a boundary container in this topology. Consequently, the complete set of boundary-crossing flows in this variant is directly visible from the DFD structure and provides the enumeration scope for the STRIDE analysis that follows [8].

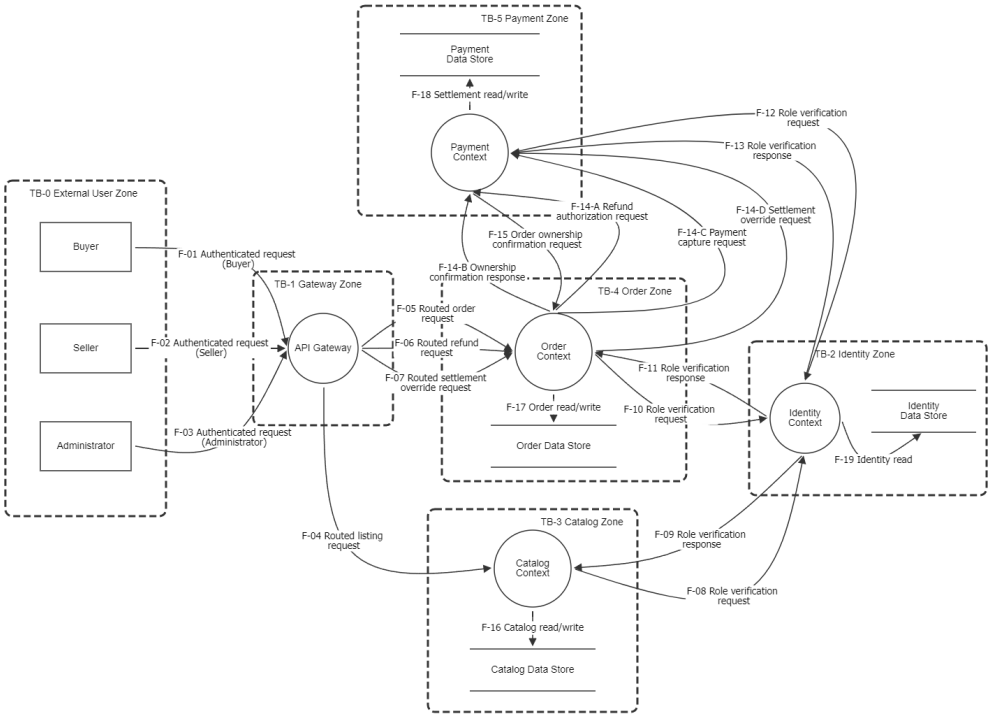


Figure 4.1: System-level DFD for the DDD-aligned variant.

4.2.3 STRIDE Threat Analysis

The threat analysis that follows applies the STRIDE-per-element procedure to the per-operation authorization flow diagrams derived from the system-level DFD above, restricted to the three in-scope categories, Elevation of Privilege, Spoofing, and Tampering, as defined in Section 3.3.5 [9], [11]. Because no two backend services share a boundary container in this variant, every inter-context flow is a boundary-crossing flow subject to full enumeration, while service-to-own-data-store flows are intra-container and are excluded from enumeration. Elevation of Privilege is not enumerated on flows whose sole function is to return a role confirmation from the Identity Context to the requesting context, since such a response does not constitute a claim independently usable for privilege escalation at the design level. Spoofing and Tampering remain applicable on those same response flows. A checkmark in the tables that follow indicates that the threat category is structurally applicable to that flow under the STRIDE-per-element procedure, and whether the applicable threat is mitigated in this variant depends on the enforcement structure described in the surrounding prose, namely the grounded PEP instances and the cross-boundary verification calls that ground their decisions in authoritative state.

4.2.3.1 OP-1: Seller Listing Publication

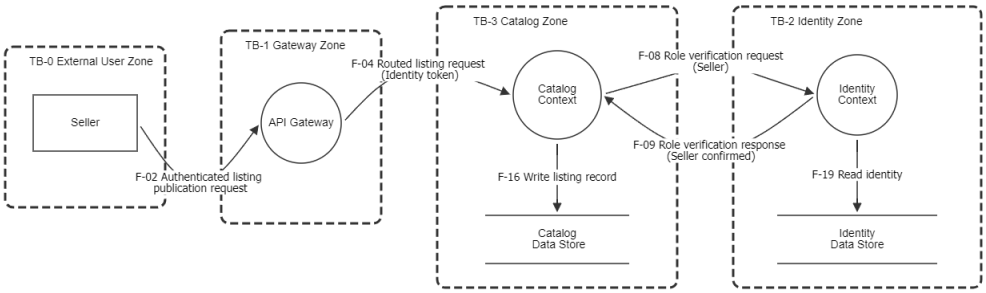


Figure 4.2: Operation-level DFD for OP-1 in the DDD-aligned variant.

In OP-1, the Seller principal submits a listing publication request to the API Gateway via flow F-02, crossing TB-0 to TB-1. The API Gateway verifies the token signature and routes the request to the Catalog Context via flow F-04, crossing TB-1 to TB-3. The Catalog Context PEP does not accept the role assertion in the forwarded token as a sufficient authorization condition and performs an independent role verification call to the Identity Context via flow F-08, crossing TB-3 to TB-2, and waits for the response on flow F-09 before admitting the request to domain logic. Only after the Identity Context has confirmed the Seller role against its Identity Data Store does the Catalog Context PEP permit the listing write to its Catalog Data Store on flow F-16.

The primary Elevation of Privilege threat in this operation is the scenario in which a Buyer-role principal submits a listing publication request and the Catalog Context admits the write by accepting the forwarded role assertion in the request payload without cross-boundary re-verification against the Identity Context. This threat is enumerated at the TB-1 to TB-3 boundary crossing on flow F-04, where an attacker holding a Buyer-role token could attempt to trigger the listing write by exploiting the absence of a grounded PEP check [9]. The Spoofing threat on flow F-08 is enumerated at the TB-3 to TB-2 boundary, since an attacker who could impersonate the Identity Context on that channel could cause the Catalog Context to accept a false role confirmation. The Tampering threat on flow F-04 is enumerated at the TB-1 to TB-3 boundary, since modification of the role claim in transit could cause the context to evaluate an incorrect role. On the response flow F-09, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since the role confirmation returned by the Identity Context on that flow does not constitute a claim independently exercisable for privilege escalation at the design level. The identified threats for OP-1 are summarized in Table 4.4.

Table 4.4: STRIDE for OP-1 in the DDD-aligned variant.

Flow ID	Elevation of Privilege	Spoofing	Tampering
F-02	✓	✓	✓
F-04	✓	✓	✓
F-08	✓	✓	✓
F-09	—	✓	✓
F-16	—	—	—

4.2.3.2 OP-2: Order Placement with Payment Capture

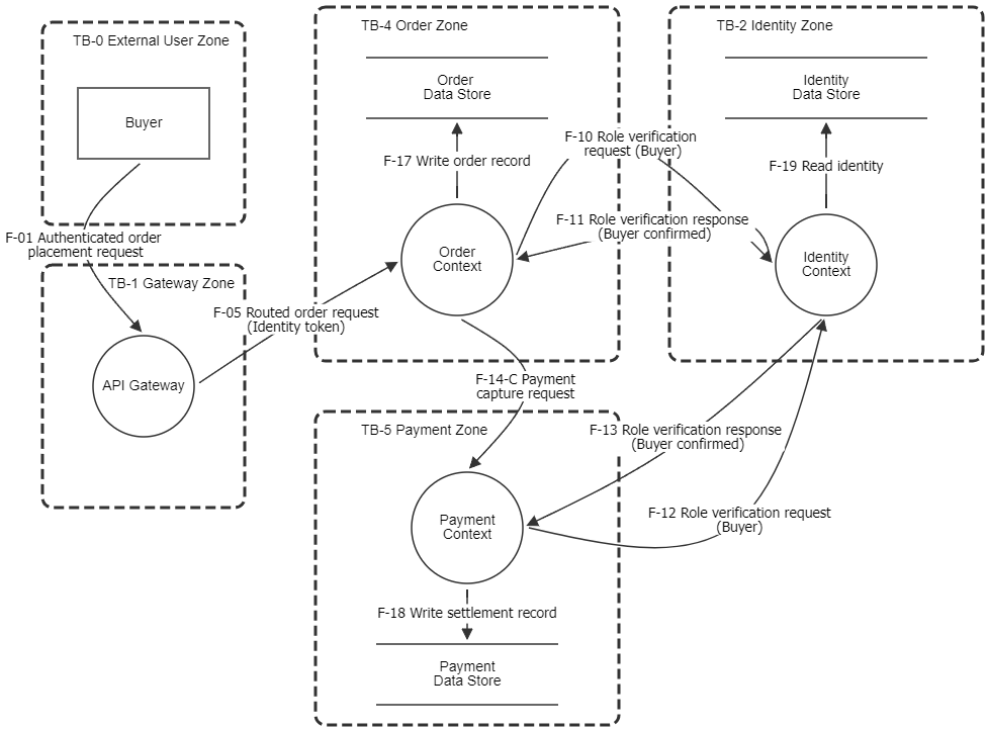


Figure 4.3: Operation-level DFD for OP-2 in the DDD-aligned variant.

In OP-2, the Buyer principal submits an order placement request to the API Gateway via flow F-01, crossing TB-0 to TB-1. The API Gateway routes the request to the Order Context via flow F-05, crossing TB-1 to TB-4. The Order Context PEP independently verifies the Buyer role by querying the Identity Context via flow F-10, crossing TB-4 to TB-2, and waits for the role confirmation on flow F-11 before writing the order record to its Order Data Store on the intra-container flow F-17. The Order Context then initiates a

payment capture call to the Payment Context via flow F-14-C, crossing TB-4 to TB-5. The Payment Context PEP performs its own independent role verification call to the Identity Context via flow F-12, crossing TB-5 to TB-2, and waits for the response on flow F-13 before admitting the settlement write to its Payment Data Store on the intra-container flow F-18. This two-stage structure means that both the Order Context and the Payment Context each ground their authorization decisions in a direct call to the authoritative Identity Data Store rather than in a role assertion forwarded across the TB-4 to TB-5 boundary.

The primary Elevation of Privilege threat in this operation concerns the TB-4 to TB-5 boundary crossing on flow F-14-C. At that crossing, the Payment Context PEP receives a payment capture request that already carries a role assertion forwarded by the Order Context. An attacker who has obtained a Buyer-role token could attempt to exploit a Payment Context implementation that accepts this forwarded assertion in place of performing its own verification call, thereby causing a settlement write to proceed under a role that the Payment Context has never independently confirmed [9]. This threat is structurally distinct from OP-1 since it involves a second context accepting a privilege claim propagated from a peer context rather than one arriving directly from the API Gateway.

The Spoofing threat on flow F-10 is enumerated at the TB-4 to TB-2 boundary crossing, since an attacker able to impersonate the Identity Context on that channel could cause the Order Context to accept a fabricated role confirmation and admit the order write. The Spoofing threat on flow F-12 is enumerated independently at the TB-5 to TB-2 boundary crossing for the same reason in the context of the Payment Context verification call. The Tampering threat on flow F-05 is enumerated at the TB-1 to TB-4 boundary crossing, since modification of the role claim in transit before the Order Context PEP evaluates it could cause the order to be admitted for an identity whose actual role is insufficient. The Tampering threat on flow F-14-C is enumerated at the TB-4 to TB-5 boundary crossing, since alteration of the role assertion carried in that inter-context request could cause the Payment Context to evaluate an incorrect role even if it does perform a verification call, because the altered claim could influence how the verification request is constructed [3]. On the response flow F-11 and on the response flow F-13, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since the role confirmations returned by the Identity Context on those flows do not constitute claims independently exercisable for privilege escalation at the design level. The intra-container flows F-17 and F-18 are excluded from enumeration. Threats enumerated for

OP-2 are summarized in Table 4.5 below.

Table 4.5: STRIDE for OP-2 in the DDD-aligned variant.

Flow ID	Elevation of Privilege	Spoofing	Tampering
F-01	✓	✓	✓
F-05	✓	✓	✓
F-10	✓	✓	✓
F-11	—	✓	✓
F-14-C	✓	✓	✓
F-12	✓	✓	✓
F-13	—	✓	✓
F-17	—	—	—
F-18	—	—	—

4.2.3.3 OP-3: Return Request and Refund Authorization

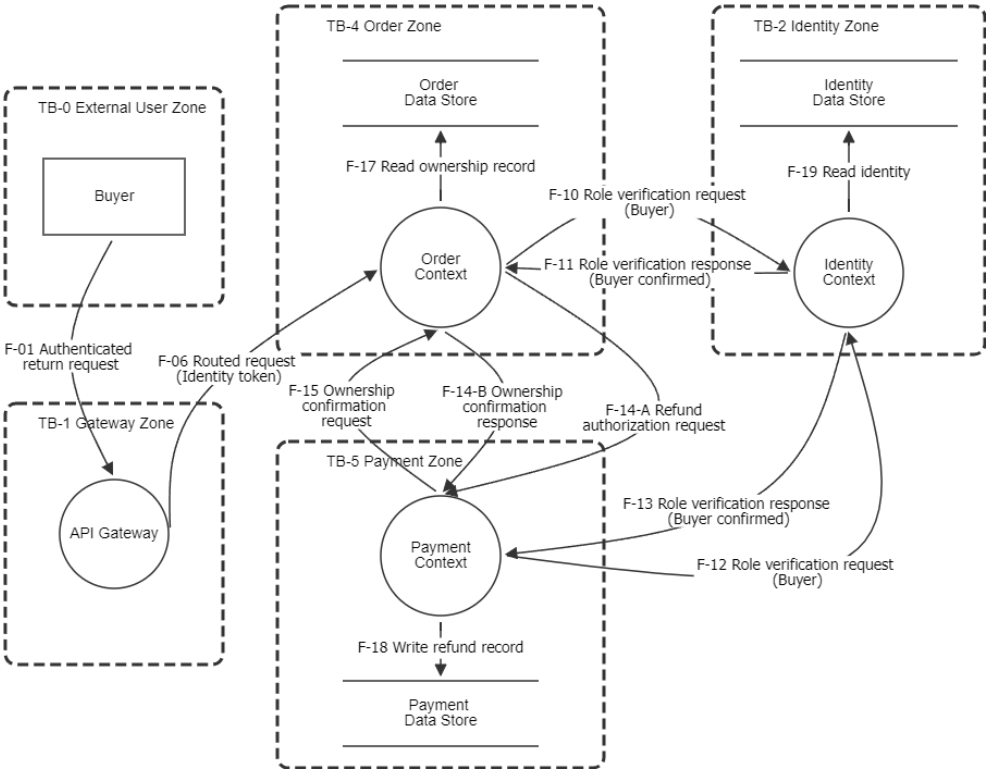


Figure 4.4: Operation-level DFD for OP-3 in the DDD-aligned variant.

In OP-3, the Buyer principal submits a return request to the API Gateway via flow F-01, crossing TB-0 to TB-1. The API Gateway routes the request to the Order Context via flow F-06, crossing TB-1 to TB-4. The Order Context PEP independently verifies the Buyer role via flow F-10 and waits for the confirmation on flow F-11. The Order Context then reads the order ownership record from its own Order Data Store on intra-container flow F-17, establishing that the requesting Buyer principal owns the targeted order, and forwards the refund authorization request to the Payment Context via flow F-14-A, crossing TB-4 to TB-5. The Payment Context PEP independently re-verifies the Buyer role via flow F-12 and receives the confirmation on flow F-13. Because the Payment Context does not own the order record, it then issues an ownership confirmation request to the Order Context via flow F-15, crossing TB-5 to TB-4, and treats the response from the Order Context as the authoritative ownership record rather than relying on the ownership context carried in flow F-14-A.

The primary Elevation of Privilege threat in OP-3 concerns the authorization of a refund for an order that the requesting Buyer principal does not own, and it operates across two structurally distinct attack points rather than a single role-check boundary. Flow F-14-A, which crosses the TB-4 to TB-5 boundary, carries the ownership assertion forwarded by the Order Context to the Payment Context, and an attacker who can craft or manipulate the payload of that flow could cause the Payment Context to initiate a refund against an order record belonging to a different user. Flow F-15, which crosses the TB-5 to TB-4 boundary in the reverse direction, carries the ownership confirmation request that the Payment Context issues to the Order Context, and an attacker who can interfere with the response returned on flow F-14-B could cause the Payment Context to accept a false ownership confirmation and authorize the refund independently of the actual ownership state held in the Order Data Store [9]. This two-point structure makes OP-3 the only operation in this variant in which the critical Elevation of Privilege risk is distributed across an outbound request flow and a cross-context verification flow.

The Spoofing threat on flow F-10 is enumerated at the TB-4 to TB-2 boundary crossing, since impersonation of the Identity Context on that channel could cause the Order Context to admit the return request for an attacker whose actual role is not Buyer. The Spoofing threat on flow F-12 is enumerated at the TB-5 to TB-2 boundary crossing for the same reason in the context of the Payment Context role verification. The Spoofing threat on flow F-15 is additionally enumerated at the TB-5 to TB-4 boundary crossing, since an attacker impersonating the Order Context on that channel could deliver a

fabricated ownership confirmation to the Payment Context. The Tampering threat on flow F-06 is enumerated at the TB-1 to TB-4 boundary crossing, since alteration of the request payload before the Order Context PEP evaluates it could cause the context to initiate a return for an order the attacker does not own. The Tampering threat on flow F-14-A is enumerated at the TB-4 to TB-5 boundary crossing, since modification of the ownership context forwarded in that flow is the mechanism by which the first Elevation of Privilege point above can be exploited [3]. On the response flow F-11 and on the response flow F-13, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since those flows carry only role confirmations from the Identity Context that do not constitute claims independently exercisable for privilege escalation. The intra-container flow F-17 is excluded from enumeration. Threats for each flow in OP-3 are shown in Table 4.6.

Table 4.6: STRIDE for OP-3 in the DDD-aligned variant.

Flow ID	**Elevation of Privilege**	**Spoofing**	**Tampering**
F-01	✓	✓	✓
F-06	✓	✓	✓
F-10	✓	✓	✓
F-11	—	✓	✓
F-12	✓	✓	✓
F-13	—	✓	✓
F-14-A	✓	✓	✓
F-14-B	—	✓	✓
F-15	✓	✓	✓
F-17	—	—	—

4.2.3.4 OP-4: Administrative Dispute Override

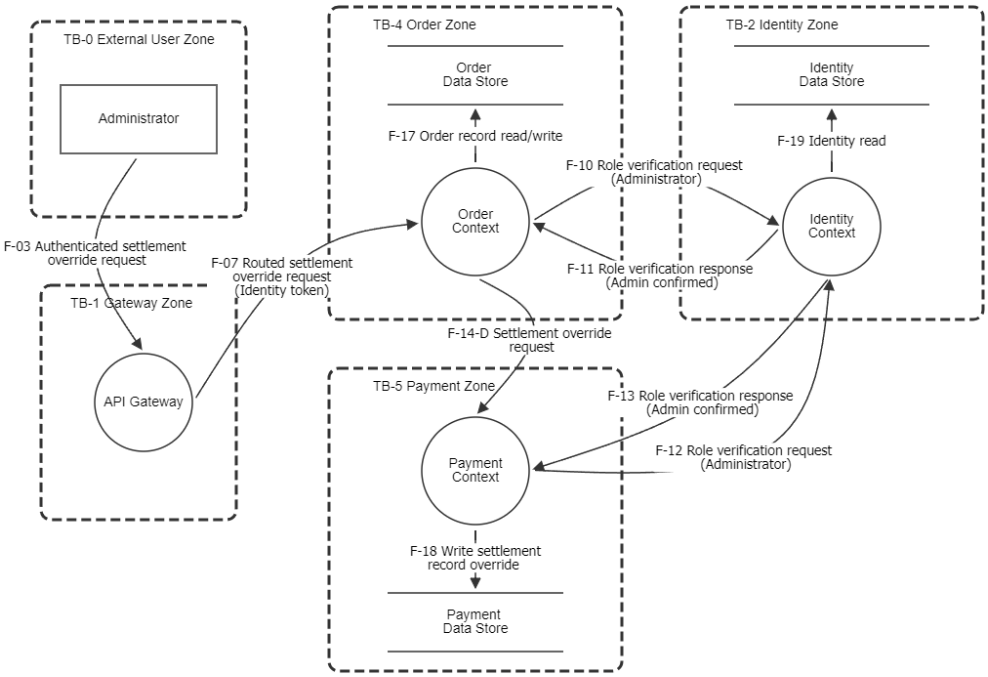


Figure 4.5: Operation-level DFD for OP-4 in the DDD-aligned variant.

In OP-4, the Administrator principal submits a settlement override request to the API Gateway via flow F-03, crossing TB-0 to TB-1. The API Gateway routes the request to the Order Context via flow F-07, crossing TB-1 to TB-4. The Order Context PEP independently verifies the Administrator role via flow F-10 and waits for the confirmation on flow F-11 before modifying the dispute state in its Order Data Store. The Order Context then forwards the settlement override request to the Payment Context via flow F-14-D, crossing TB-4 to TB-5. The Payment Context PEP independently re-verifies the Administrator role via flow F-12 and waits for the response on flow F-13 before executing the irreversible settlement write to its Payment Data Store. Both verification calls are directed to the Identity Context because the Administrator role is defined and controlled exclusively within the Identity Data Store, which means that neither context can evaluate the privilege level of the caller without crossing TB-2.

The primary Elevation of Privilege threat in OP-4 is the highest-privilege scenario in the attacker model, in which a Buyer-role principal causes an irreversible settlement write to be executed by the Payment Context without holding the Administrator role. What makes this operation structurally

distinct from OP-1 and OP-2 is that the consequence of a failed role check is not admission of a domain write to the requesting context alone but rather propagation of an Administrator-scoped action across the TB-4 to TB-5 boundary into the Payment Context, where the write is both irreversible and financially significant. The Elevation of Privilege threat is therefore enumerated at both the TB-1 to TB-4 boundary crossing on flow F-07 and the TB-4 to TB-5 boundary crossing on flow F-14-D, since each crossing is a structural point at which a PEP must independently confirm the Administrator role before proceeding [2], [9].

The Spoofing threat on flow F-10 is enumerated at the TB-4 to TB-2 boundary crossing. Because the Administrator role governs the most sensitive write path in the system, an attacker able to impersonate the Identity Context on that channel could cause the Order Context to accept a fabricated Administrator confirmation and forward the override request to the Payment Context without a legitimate role basis. The Spoofing threat on flow F-12 is enumerated independently at the TB-5 to TB-2 boundary crossing, since the Payment Context's verification call is a separate channel that can be targeted regardless of whether the Order Context check has already succeeded. The Tampering threat on flow F-07 is enumerated at the TB-1 to TB-4 boundary crossing, since modification of the role claim in that request before the Order Context PEP evaluates it could cause the Order Context to construct and forward a settlement override request to the Payment Context without a valid Administrator role being present. The Tampering threat on flow F-14-D is enumerated at the TB-4 to TB-5 boundary crossing, since alteration of the role assertion carried in the inter-context request could influence how the Payment Context constructs its own verification call to the Identity Context, potentially causing it to verify the wrong role or to skip constructing the call entirely [3]. On the response flow F-11 and on the response flow F-13, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since the Administrator role confirmations returned by the Identity Context on those flows do not themselves grant access and cannot be independently exercised by an attacker for privilege escalation at the design level. All enumerated threats across OP-1 through OP-4 are carried forward to Chapter 5. The specific threat breakdown for OP-4 is given in Table 4.7.

Table 4.7: STRIDE for OP-4 in the DDD-aligned variant.

Flow ID	Elevation of Privilege	Spoofing	Tampering
F-03	✓	✓	✓
F-07	✓	✓	✓
F-10	✓	✓	✓
F-11	—	✓	✓
F-14-D	✓	✓	✓
F-12	✓	✓	✓
F-13	—	✓	✓

4.3 The Baseline Variant

The baseline variant decomposes the e-commerce marketplace domain into four independently deployable backend services, organized according to a technical workflow grouping strategy rather than a domain-ownership strategy. The API Gateway and the Identity Service each occupy their own dedicated trust boundary, while the Order Service, the Catalog Service, and the Payment Service are co-located within a single shared boundary container designated TB-3, referred to throughout this section as the Business Logic boundary. The domain scope, the set of privileged operations, and the attacker model are identical to those applied to the variant described in Section 4.2.

4.3.1 Service Decomposition

The baseline variant organizes the e-commerce marketplace into four distinct boundary containers. The External User boundary container, TB-0, contains the three actor roles, Buyer, Seller, and Administrator, represented as external entities. The API Gateway boundary container, TB-1, contains the API Gateway, which is responsible for edge-level authentication verification and request routing. The Identity boundary container, TB-2, contains the Identity Service and its co-located Identity Data Store. The Identity Service holds sole responsibility for maintaining the authoritative role and membership records for all principals and for issuing and validating the identity tokens consumed by other services. The Business Logic boundary container, TB-3, contains three services together with their respective co-located data stores, namely the Catalog Service and Catalog Data Store, the Order Service and Order Data Store, and the Payment Service and Payment Data Store.

The Catalog Service holds responsibility for managing product listing records and for enforcing role constraints on listing publication and modification operations. The Order Service holds responsibility for managing order records and the ownership relationships associated with them. The Payment Service holds responsibility for managing settlement and refund records. Each service owns its data store exclusively, and all three data stores reside within the same shared boundary container as the services that own them.

The grouping of the Catalog Service, the Order Service, and the Payment Service within a single shared boundary container reflects a technical workflow grouping strategy, in which services that participate in the same transactional sequence are co-located rather than separated by domain. The practical consequence is that the Order Service and the Payment Service are co-located within TB-3 because the payment capture step is modeled as a continuation of the same transactional workflow as order placement, rather than as a cross-domain interaction between two independently authoritative services. Richardson identifies subdomain-based decomposition as the preferred alternative, noting that organizing services by bounded context avoids conflating domain responsibilities that have distinct ownership semantics [41]. This arrangement is an instance of what NIST SP 800-204 and the OWASP identify as the perimeter-trust model, a documented pattern found in operational deployments where teams have not yet adopted a zero-trust internal networking posture, in which services sharing a deployment tier communicate over an implicitly trusted internal channel on the premise that the gateway has already authenticated and coarsely filtered the incoming request [2], [3], [19].

NIST SP 800-204 explicitly warns that reliance on gateway-level authentication as the sole authorization enforcement point creates a structural gap at the individual service level, because fine-grained authorization decisions require business-domain context that the gateway cannot and should not possess [3]. OWASP similarly identifies this pattern as one of the root causes of the authorization consistency problem in microservice architectures, recommending a defense-in-depth model in which authorization is applied at the edge, at the service boundary, and within the domain logic of each service [2]. The baseline variant in this study deliberately models the perimeter-trust assumption in order to stress-test its consequences for authorization enforcement consistency under the defined attacker model, and is not presented as a recommended practice, but as a realistic architectural assumption whose structural effects on the DFD topology and the STRIDE

enumeration results are evaluated in Chapter 5.

The perimeter-trust consolidation is one of several non-DDD-aligned baselines that could have been modeled. A layered decomposition that separates the Catalog, Order, and Payment services into distinct deployable units while requiring each to enforce its own authorization locally, or a CRUD-per-entity decomposition, in which each persistent entity is wrapped by its own service, would both represent a stronger baseline, since they preserve fine-grained enforcement obligations without aligning service boundaries to bounded contexts [41]. Either of those baselines would likely narrow the metric deltas reported in Chapter 5, because each retains some of the per-service enforcement structure that the perimeter-trust consolidation removes. However, the configuration adopted here was chosen because it isolates the structural effect being studied, by making the boundary alignment strategy the only distinguishing design variable.

The decomposition preserves the same functional scope across all four privileged operations. The Identity Service retains sole authority over role and membership records, and all business-capability services depend on it for authorization verification. The trust boundary placement is the parameter that distinguishes this variant, where a single shared boundary is drawn around the three business services rather than isolating each service within its own boundary. As a consequence, the authorization enforcement obligations of the Order Service, the Catalog Service, and the Payment Service with respect to one another are not structurally enforced by a trust-boundary crossing at the DFD level, and the enforcement of those obligations depends on each service's internal logic rather than on the structural visibility of the flow as a boundary-crossing event in the threat model [3], [8].

4.3.2 Data Flow Diagram

The system-level DFD for the baseline variant is constructed by applying the modeling conventions defined in Section 3.3.4 to the decomposition described in Section 4.3.1. Four trust boundary containers are defined for this variant. TB-0 contains the external user population, represented as the Buyer, Seller, and Administrator external entities. TB-1 contains the API Gateway, which performs edge-level authentication verification and routes incoming requests to the appropriate backend service. TB-2 contains the Identity Service and its co-located Identity Data Store. TB-3 is the shared Business Logic boundary container, with the Catalog Service and its Catalog Data Store, the Order Service and its Order Data Store, and the Payment Service, as well as its

Payment Data Store.

The trust boundary topology shown in Figure 4.6 reflects the perimeter-trust assumption described in Section 4.3.1. The structural consequence of this assumption is directly visible in the DFD, where the three business-capability services share a single boundary container, and all flows between them, including those that carry authorization-relevant context between functionally distinct service areas, are intra-container flows within TB-3. There are no internal trust boundaries between the Catalog Service, the Order Service, and the Payment Service in this topology.

Under the modeling conventions of Section 3.3.4, any flow whose source and destination reside within the same boundary container is an intra-container flow and is not subject to STRIDE-per-element enumeration. All flows between the three business services, as well as all flows between those services and their respective co-located data stores, are therefore intra-container flows in this variant. The complete set of trust-boundary crossings is limited to flows between TB-0 and TB-1, between TB-1 and TB-2, between TB-1 and TB-3, and between TB-2 and TB-3. No inter-service flow within TB-3 constitutes a trust-boundary crossing, and the service-to-service interactions that implement the authorization logic of OP-2, OP-3, and OP-4 are not subject to STRIDE enumeration as boundary-crossing events within the Business Logic boundary container.

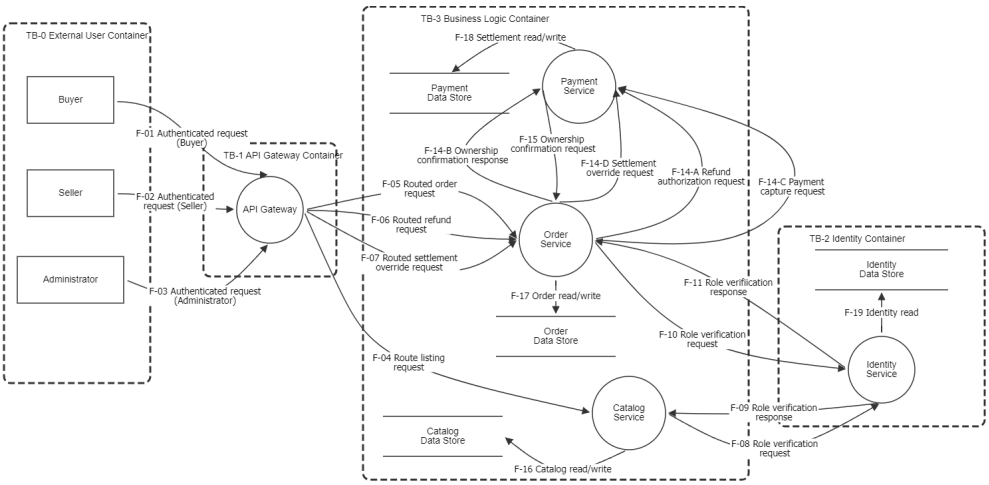


Figure 4.6: System-level DFD for the baseline variant.

The absence of internal trust boundaries within TB-3 means that the authorization enforcement structure for OP-2, OP-3, and OP-4, each of which involves a payment processing step initiated by the Order Service, is not

visible as a set of boundary-crossing events at the DFD level. The Payment Service receives requests from the Order Service via intra-container flows and, under the perimeter-trust model, relies on the Order Service having already completed the relevant authorization check before initiating those calls. The structural implications of this condition for the STRIDE enumeration and the evaluation metrics are described in the per-operation analysis that follows [3], [8].

4.3.3 STRIDE Threat Analysis

The STRIDE analysis for the baseline variant applies the STRIDE-per-element procedure defined in Section 3.3.5 to the system-level DFD described in Section 4.3.2 and to the per-operation authorization flow diagrams presented in this section. The analysis is restricted to the three in-scope threat categories, Elevation of Privilege, Spoofing, and Tampering, and is scoped to the trust-boundary crossings and flows that participate in the authorization of the four privileged operations under the defined attacker model [9], [11]. The remaining STRIDE categories, Repudiation, Information Disclosure, and Denial of Service, fall outside the authorization-relevant subset defined in Section 3.3.5 and are excluded from enumeration across all four operations. This exclusion applies uniformly to every flow in the analysis and is consistent with the DFD modeling conventions established in Section 3.3.4. Intra-container flows are similarly excluded from STRIDE-per-element enumeration throughout, because they do not cross a trust boundary and therefore do not constitute boundary-crossing events under those conventions. Their presence in the STRIDE applicability tables with all categories marked as not applicable confirms that they were evaluated and excluded on methodological grounds.

A structurally important property of the baseline variant that shapes the enumeration results across three of the four operations is the consolidation of the Catalog Service, the Order Service, and the Payment Service within TB-3. Because flows between those services are intra-container, the STRIDE-per-element procedure does not enumerate authorization threats against them as boundary-crossing flows. Any authorization failure that arises on those flows is consequently not captured as a trust-boundary-crossing threat in the enumeration. The structural condition under which such a failure could materialize, specifically the absence of an independent authorization check at the Payment Service when it receives a request from the Order Service, is classified as implicit trust within TB-3 rather than as a gap at a visible boundary crossing. This classification is reflected in the M2 count defined

in Section 3.5 and its implications are analyzed in Chapter 5.

In the absence of any design-level specification of transport security mechanisms in the DFD, all three in-scope threat categories are treated as applicable at every trust-boundary crossing unless a structural condition eliminates one of them for a specific flow type. One such condition applies to response flows whose sole function is to return a role confirmation to the service that initiated the verification request. For such flows Elevation of Privilege is not enumerated, because the response does not carry a claim that an attacker could independently exercise to gain elevated access at the design level. Spoofing and Tampering remain applicable on response flows, since a compromised Identity Service could return a false confirmation, and the response payload could be modified in transit to misrepresent the role decision [8], [9].

4.3.3.1 OP-1: Seller Listing Publication

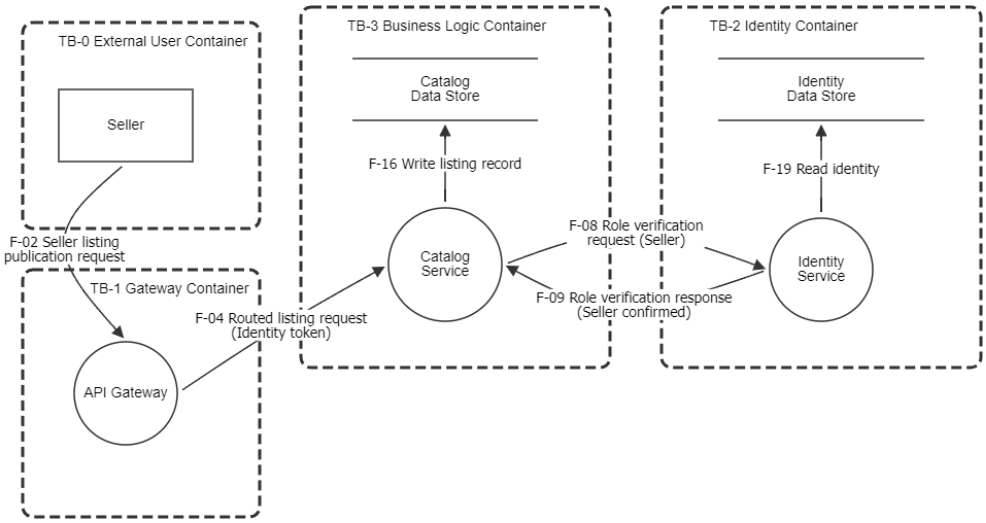


Figure 4.7: Operation-level DFD for OP-1 in the baseline variant.

In OP-1, the Seller principal submits a listing publication request to the API Gateway via flow F-02, crossing TB-0 to TB-1. The API Gateway verifies the token signature and routes the request to the Catalog Service via flow F-04, crossing TB-1 to TB-3. The Catalog Service PEP does not accept the role assertion in the forwarded token as a sufficient authorization condition and performs an independent role verification call to the Identity Service via flow F-08, crossing TB-3 to TB-2, before admitting the request to its domain logic.

The Identity Service responds on flow F-09, and upon confirmation of the Seller role, the Catalog Service completes the write to its Catalog Data Store on the intra-container flow F-16. The PEP instance at the Catalog Service is therefore grounded, since the authorization decision is based on Identity Data Store state consulted through a cross-boundary verification call rather than on the role claim propagated in the forwarded token alone.

OP-1 does not involve any interaction between the Catalog Service and the other business services within TB-3. All flows in this operation that reach a service outside TB-3 cross an explicit trust boundary, and the authorization flow structure is therefore fully captured by the STRIDE-per-element enumeration.

The primary Elevation of Privilege threat is the submission of a Buyer-role token in a listing publication request, with the attacker relying on the Catalog Service accepting the forwarded role assertion without cross-boundary re-verification. This threat is structurally mitigated because the Catalog Service PEP performs an explicit verification call to the Identity Service via a trust-boundary crossing, and the response from TB-2 is the condition for admission to domain logic. The Spoofing threat on flow F-08, in which an adversary intercepts or substitutes the role verification request, is enumerated at the TB-3 to TB-2 crossing. The Tampering threat on flow F-04, in which the identity token is modified in transit across TB-1 to TB-3, is similarly enumerated. On the response flow F-09, Spoofing and Tampering are enumerated because the confirmation payload could be forged or modified in transit, but Elevation of Privilege is not enumerated, since the response carries no claim that an attacker could exercise independently to gain elevated access. All enumerated threats for OP-1 are summarized in Table 4.8.

Table 4.8: STRIDE for OP-1 in the baseline variant.

Flow ID	**Elevation of Privilege**	**Spoofing**	**Tampering**
F-02	✓	✓	✓
F-04	✓	✓	✓
F-08	✓	✓	✓
F-09	—	✓	✓
F-16	—	—	—

4.3.3.2 OP-2: Order Placement with Payment Capture

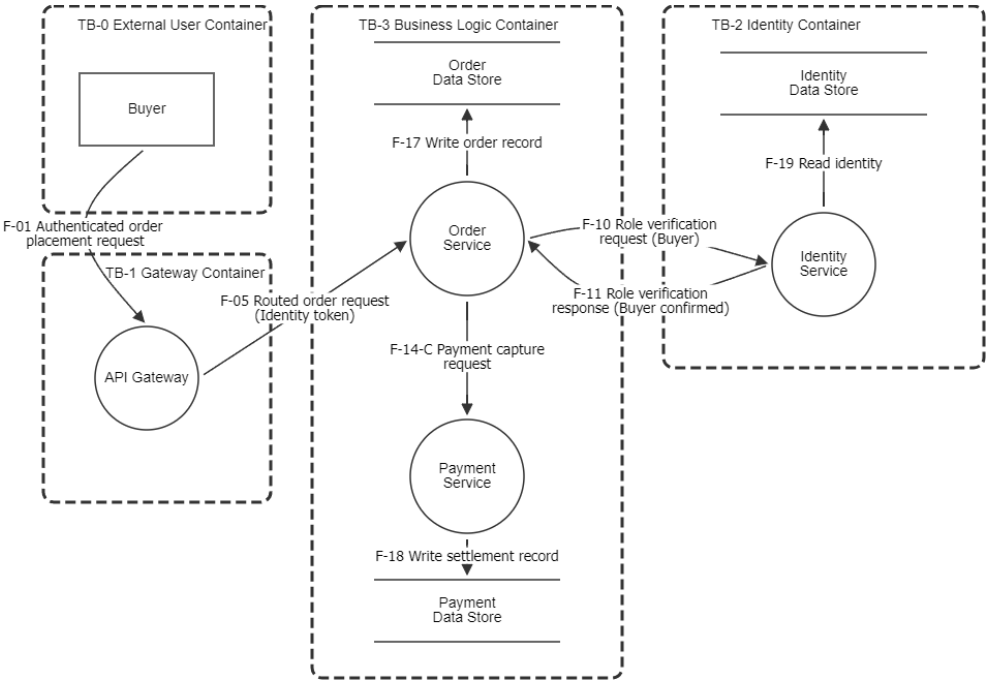


Figure 4.8: Operation-level DFD for OP-2 in the baseline variant.

In OP-2, the Buyer principal submits an order placement request to the API Gateway via flow F-01, crossing TB-0 to TB-1. The API Gateway routes the request to the Order Service via flow F-05, crossing TB-1 to TB-3. The Order Service PEP independently verifies the Buyer role by querying the Identity Service via flows F-10 and F-11, which cross TB-3 to TB-2 and TB-2 to TB-3 respectively. Upon receiving the confirmation, the Order Service writes the order record to the Order Data Store via the intra-container flow F-17 and then initiates a payment capture call to the Payment Service via the intra-container flow F-14-C. Because the Order Service and the Payment Service both reside within TB-3, flow F-14-C does not cross a trust boundary. The Payment Service consequently receives the payment capture request without a structural enforcement mechanism requiring it to perform an independent role verification before processing the settlement write.

The Payment Service does not host a grounded PEP instance for this operation, because the intra-container flow structure removes the structural condition, namely a trust-boundary crossing, at which the DFD modeling conventions require a grounded PEP to be present. The authorization decision

for the settlement step is therefore made entirely within TB-3, based on the implicit trust that the Order Service has already completed the required authorization check. Whether the Payment Service's internal logic performs an additional check is a question of implementation detail that falls outside the scope of design-level analysis [8], [9].

The primary Elevation of Privilege threat is the scenario in which a Buyer-role principal places an order and the payment capture proceeds to settlement without the Payment Service performing an independent role verification against the authoritative Identity Data Store. The structural condition that enables this, namely the absence of a grounded PEP at the Payment Service, is classified as an implicit-trust flow within TB-3 and contributes to the metric count defined in Section 3.5. The Tampering threat on flow F-05 and the Spoofing threat on flow F-10 are enumerated at their respective boundary crossings. On the response flow F-11, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since the confirmation returned by the Identity Service does not constitute a claim that an attacker could exercise independently for privilege escalation. The intra-container flow F-14-C is excluded from enumeration, consistent with the scope defined in Section 3.3.5. Threats enumerated for OP-2 are shown in Table 4.9.

Table 4.9: STRIDE for OP-2 in the baseline variant.

Flow ID	**Elevation of Privilege**	**Spoofing**	**Tampering**
F-01	✓	✓	✓
F-05	✓	✓	✓
F-10	✓	✓	✓
F-11	—	✓	✓
F-14-C	—	—	—
F-17	—	—	—

4.3.3.3 OP-3: Return Request and Refund Authorization

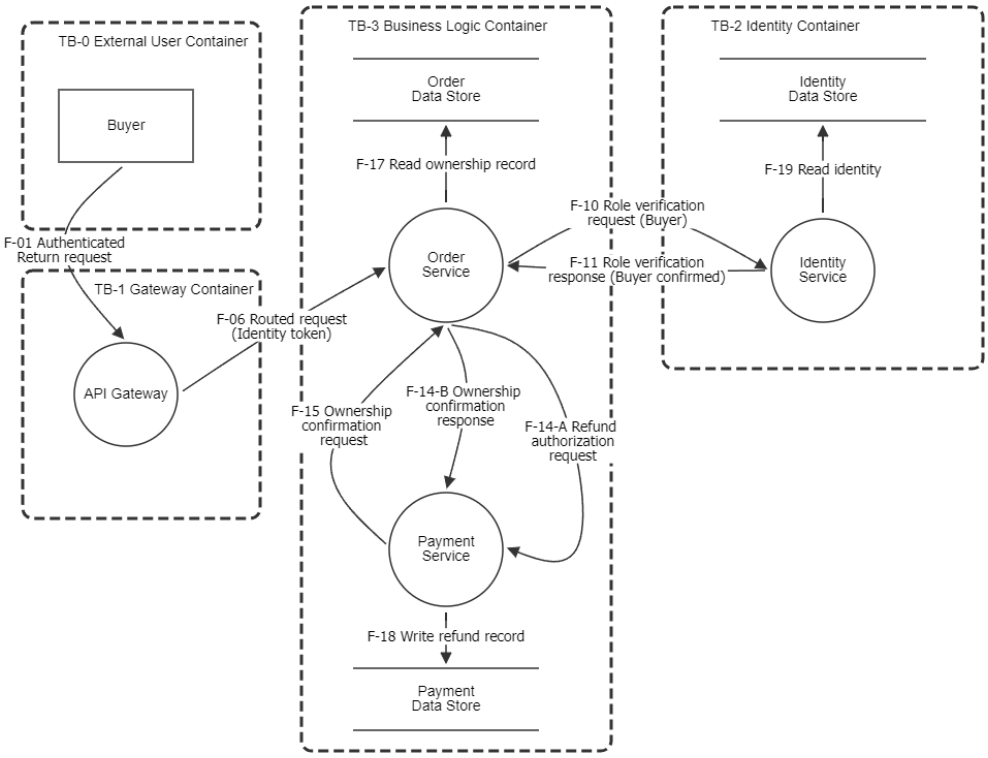


Figure 4.9: Operation-level DFD for OP-3 in the baseline variant.

In OP-3, the Buyer principal submits a return request to the API Gateway via flow F-01, crossing TB-0 to TB-1. The API Gateway routes the request to the Order Service via flow F-06, crossing TB-1 to TB-3. The Order Service PEP independently verifies the Buyer role by querying the Identity Service via flows F-10 and F-11, and then reads the order ownership record from the Order Data Store via the intra-container flow F-17, confirming that the requesting principal owns the targeted order. The Order Service then forwards the refund authorization request to the Payment Service via the intra-container flow F-14-A, carrying the verified ownership context as part of the request payload. The Payment Service initiates an ownership confirmation request back to the Order Service via the intra-container flow F-15 and receives the response on intra-container flow F-14-B, before writing the refund record to its Payment Data Store.

OP-3 presents the most structurally significant case for the ownership-verification authorization challenge type. The flows carrying ownership context between the Order Service and the Payment Service, including the

forwarded claim in F-14-A and the confirmation exchange on F-14-B and F-15, are entirely intra-container, and those flows are therefore invisible to the STRIDE-per-element procedure as boundary-crossing events, consistent with the scope defined in Section 3.3.5.

The primary Elevation of Privilege threat is the scenario in which the Payment Service processes a refund for an order not owned by the requesting Buyer principal, by accepting the ownership context forwarded by the Order Service on flow F-14-A without independently verifying it against a source it owns or controls. This threat is structurally present because the intra-container nature of flows F-14-A, F-14-B, and F-15 removes the boundary-crossing enforcement mechanism that would require independent ownership verification at the Payment Service. It is therefore classified as an implicit-trust flow condition within TB-3 [2], [3]. The Spoofing threat on flow F-10 and the Tampering threat on flow F-06 are enumerated at their respective boundary crossings. On the response flow F-11, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since the role confirmation returned on that flow does not carry a claim independently exercisable for privilege escalation. Summary of enumerated threats for OP-3 is shown in Table 4.10.

Table 4.10: STRIDE for OP-3 in the baseline variant.

Flow ID	**Elevation of Privilege**	**Spoofing**	**Tampering**
F-01	✓	✓	✓
F-06	✓	✓	✓
F-10	✓	✓	✓
F-11	—	✓	✓
F-14-A	—	—	—
F-14-B	—	—	—
F-15	—	—	—
F-17	—	—	—

4.3.3.4 OP-4: Administrative Dispute Override

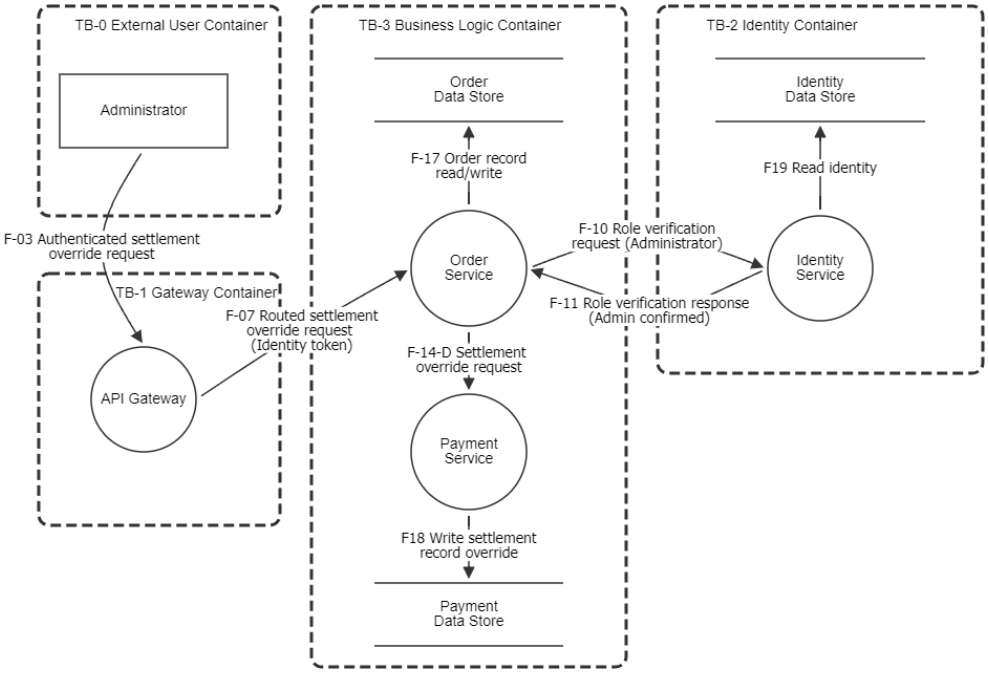


Figure 4.10: Operation-level DFD for OP-4 in the baseline variant.

In OP-4, the Administrator principal submits a dispute override request to the API Gateway via flow F-03, crossing TB-0 to TB-1. The API Gateway routes the request to the Order Service via flow F-07, crossing TB-1 to TB-3. The Order Service PEP independently verifies the Administrator role by querying the Identity Service via flows F-10 and F-11, which cross TB-3 to TB-2 and TB-2 to TB-3 respectively. Upon receiving the Administrator role confirmation, the Order Service modifies the order record in its Order Data Store via an intra-container flow and then forwards the settlement override request to the Payment Service via the intra-container flow F-14-D, carrying the role context in the request payload. The Payment Service, residing within the same boundary container as the Order Service, processes the settlement write to its Payment Data Store without performing an independent cross-boundary role verification call to the Identity Service.

OP-4 is an irreversible operation requiring Administrator-level privilege, and the absence of an independent role verification at the Payment Service means that the downstream settlement write is authorized entirely on the basis of the role context forwarded by the Order Service via the intra-container flow F-14-D. The structural condition that enables this is the absence of a

trust-boundary crossing between the Order Service and the Payment Service within TB-3. As a result, there is no boundary-crossing point at which the DFD modeling conventions would require a grounded PEP to be present at the Payment Service for this operation. The structural gap is therefore also classified as an implicit-trust flow within TB-3 [2], [3].

The Tampering threat on flow F-07 and the Spoofing threat on flow F-10 are enumerated at their respective boundary crossings. On the response flow F-11, Spoofing and Tampering are enumerated, but Elevation of Privilege is not, since the Administrator role confirmation returned on that flow does not itself grant access and cannot be independently exercised by an attacker for privilege escalation at the design level. The intra-container flow F-14-D is excluded from enumeration, consistent with the scope defined in Section 3.3.5 and complete threat enumeration for OP-4 is summarized in Table 4.11.

Table 4.11: STRIDE for OP-4 in the baseline variant.

Flow ID	Elevation of Privilege	Spoofing	Tampering
F-03	✓	✓	✓
F-07	✓	✓	✓
F-10	✓	✓	✓
F-11	—	✓	✓
F-14-D	—	—	—

The structural consequences of the TB-3 topology for the authorization enforcement completeness of this operation, and their contribution to the evaluation metrics, are discussed in Chapter 5 [8], [9].

4.3.4 Summary

This chapter has produced the primary evidence that Chapter 5 draws on. For both variants it documented the service decomposition, the system-level DFD with its trust boundary containers, and the per-operation STRIDE enumeration. The decisive structural difference is that every inter-service delivery point in the DDD-aligned variant crosses a drawn trust boundary and carries a grounded PEP, whereas the baseline variant consolidates the Catalog, Order, and Payment services into the shared TB-3 container, so the Payment Service receives several privileged requests under implicit trust with no grounded PEP. These per-operation flow structures are carried forward into Chapter 5, where they become the counts from which M1 to M4 are computed.

Chapter 5

Results and Analysis

This chapter presents the metric results derived from the DFD and per-operation authorization flow diagrams constructed for the two architecture variants in Chapter 4, and interprets those results in relation to the research question and hypothesis stated in Chapter 1. Section 5.1 recaps the single structural difference between the two variants from which all metric variation derives. Section 5.2 examines the per-operation breakdown for each of the four privileged operations, tracing how each contributes to the aggregate counts. Section 5.3 reports the computed values for M1 through M4 and assesses each against its directional prediction, and Section 5.4 situates the findings within the related work reviewed in Chapter 2.

5.1 Structural Overview of Both Variants

Table 5.1: Structural topology summary of both architecture variants.

Property	DDD	Baseline
Trust boundary containers	6	4
Independently deployed services	4	4
Total privileged-operation request flows	7	7
Cross-boundary request flows	7	4
Intra-container request flows	0	3

The single structural difference between the two variants, from which all subsequent metric variation derives, is the assignment of trust boundary containers to backend services. In the DDD-aligned variant, the Catalog

Context, the Order Context, and the Payment Context each occupy a dedicated boundary container, namely TB-3, TB-4, and TB-5 respectively, so that every inter-service flow constitutes a trust-boundary crossing subject to the STRIDE enumeration procedure defined in Section 3.5. In the baseline variant, those same three services are consolidated within a single shared Business Logic boundary container, TB-3, so that the flows between them are intra-container exchanges invisible to the STRIDE-per-element procedure as boundary-crossing events, as documented in Section 4.3.1. This decomposition difference is the independent variable of the comparison, consistent with the service decomposition documentation in Sections 4.2.1 and 4.3.1, and it is the analytical anchor for the per-operation and aggregate analyses that follow in Sections 5.2 and 5.3, where every difference observed in M1 through M4 traces to this single boundary-placement decision rather than to any other design variable.

5.2 Per-Operation Metric Results

OP-1 produces identical metric inputs in both variants because the Catalog Service in the baseline shares TB-3 with the Order and Payment Services but OP-1 involves no further inter-service call within that shared container, leaving the boundary-crossing structure unchanged. This identity functions as an internal validity check for the measurement procedure, since structurally equivalent operations yield equivalent metric scores, and the divergence observed in OP-2, OP-3, and OP-4 can therefore be read as reflecting architectural structure rather than procedural variation. All such divergence traces to a single cause, the TB-3 consolidation in the baseline variant, which absorbs the Order and Payment Services into one shared boundary container, eliminates the trust-boundary crossing between them, and removes the structural condition under which the DFD modeling conventions require the Payment Service to host a grounded PEP for any privileged operation it receives from the Order Service.

5.2.1 OP-1: Seller Listing Publication

OP-1 is the simplest operation, with a single inter-service delivery point in both variants. The flow classification is given in Table 5.2.

Table 5.2: Flow classification for OP-1 (per Table 3.2).

Class	DDD	Baseline
Delivery	F-04	F-04
Verification	F-08, F-09	F-08, F-09
Non-inter-service	F-02, F-16, F-19	F-02, F-16, F-19

The delivery point is flow F-04, which carries the routed listing request from the Gateway boundary container (TB-1) to the Catalog context, as shown in Figures 4.2 and 4.7. In the DDD-aligned variant the Catalog Context occupies its own dedicated container (TB-3 Catalog boundary container), while in the baseline variant the Catalog Service resides in the shared Business Logic boundary container, TB-3, alongside the Order Service and the Payment Service. However, because OP-1 involves no further inter-service call within the business logic layer, the shared boundary container introduces no intra-container implicit-trust flows for this operation. Both variants produce identical metric input quantities for OP-1, and the single grounded PEP instance in each variant grounds its authorization decision through a two-crossing round-trip to the Identity context via flows F-08 and F-09. The resulting metric input quantities are summarized in Table 5.3.

Table 5.3: Metric input quantities for OP-1.

Quantity	DDD	Baseline
Inter-service delivery points participating in OP-1	1	1
Grounded PEP instances at those delivery points	1	1
Implicit-trust inter-service flows	0	0
Total privileged-operation request flows for OP-1	1	1
Number of containers with auth decision (k)	1	1
Per-operation fragmentation score ($1/k$)	1.00	1.00
Verification depth per grounded PEP	2	2

5.2.2 OP-2: Order Placement with Payment Capture

OP-2 introduces a second inter-service delivery point, and the two variants diverge in their enforcement structure for the first time. The flow classification is given in Table 5.4.

Table 5.4: Flow classification for OP-2 (per Table 3.2).

Class	DDD	Baseline
Delivery	F-05, F-14-C	F-05, F-14-C
Verification	F-10, F-11, F-12, F-13	F-10, F-11
Non-inter-service	F-01, F-17, F-18, F-19	F-01, F-17, F-18, F-19

As an example of the classification rules in Table 3.2, consider three OP-2 flows in turn. Flow F-05 carries the routed order placement request from the API Gateway to the receiving order service, which hosts a grounded PEP that verifies the Buyer role before admitting the request, so F-05 is a delivery point with grounded enforcement in both variants and adds to both the numerator and the denominator of M1. Flows F-10 and F-11 carry that PEP's role verification query to the identity service and the confirmation returned to it rather than a fresh privileged request, so they are verification flows that contribute a single round trip of depth two to M4 and play no part in M1 or M2.

Flow F-14-C, the payment capture request, is the flow whose class depends on the boundary alignment strategy. In the DDD-aligned variant it crosses from the Order Context into the Payment Context, where the Payment Context grounds its own decision through an independent verification call, so F-14-C is a covered delivery point that adds to the M1 numerator. In the baseline variant the Order Service and the Payment Service share the single container TB-3, the Payment Service applies no independent check, and the same flow F-14-C is an implicit-trust flow that adds to the M2 numerator instead. One flow is thus a delivery point under one boundary strategy and an implicit-trust flow under the other, which is exactly the structural distinction that M1 and M2 are defined to capture.

In the DDD-aligned variant, flow F-14-C crosses the TB-4 to TB-5 trust boundary, placing the Payment Context in an independent enforcement position. That service grounds its authorization decision through a direct round-trip to the Identity Context via flows F-12 and F-13 before admitting the payment capture, as shown in Figure 4.3. In the baseline variant, F-14-C is an intra-container flow within TB-3, so the Payment Service receives the payment capture request without crossing a trust boundary and applies no independent PEP. Therefore, that delivery point is counted as an implicit-trust inter-service flow, and the only grounded PEP for OP-2 is the Order Service instance that verifies the Buyer role via flows F-10 and F-11. The DDD-aligned variant consequently contributes two grounded PEPs each with a verification depth of

two, while the baseline variant contributes one grounded PEP and one implicit-trust flow. These differences are reflected in the metric input quantities in Table 5.5.

Table 5.5: Metric input quantities for OP-2.

Quantity	DDD	Baseline
Inter-service delivery points participating in OP-2	2	2
Grounded PEP instances at those delivery points	2	1
Implicit-trust inter-service flows	0	1
Total privileged-operation request flows for OP-2	2	2
Number of containers with auth decision (k)	2	1
Per-operation fragmentation score ($1/k$)	0.50	1.00
Verification depth per grounded PEP (list)	2, 2	2

5.2.3 OP-3: Return Request and Refund Authorization

OP-3 is the structurally most complex operation in both variants, as the Payment context must verify both the Buyer role and order ownership before authorizing the refund. The flow classification is given in Table 5.6.

Table 5.6: Flow classification for OP-3 (per Table 3.2).

Class	DDD	Baseline
Delivery	F-06, F-14-A	F-06, F-14-A
Verification	F-10, F-11, F-12, F-13, F-15, F-14-B	F-10, F-11,F-14-B, F-15
Non-inter-service	F-01, F-17, F-18, F-19	F-01, F-17, F-18, F-19

In the DDD-aligned variant, flow F-14-A crosses the TB-4 to TB-5 boundary, placing the Payment Context in an independent enforcement position. That PEP instance grounds its authorization through a role verification sub-path to the Identity Context (flows F-12 and F-13, depth 2) and an ownership confirmation sub-path to the Order Context (flows F-15 and F-14-B, depth 2), yielding a combined verification depth of four for that instance, as shown in Figure 4.4. In the baseline variant, flow F-14-A is an intra-container exchange within TB-3, so the refund authorization request

does not constitute a trust-boundary crossing and is counted as an implicit-trust inter-service flow. The ownership confirmation exchange (flows F-15 and F-14-B) is also intra-container, but, as in the DDD-aligned variant, constitutes a verification sub-path rather than a delivery point. Because the Payment Service is not a grounded PEP for OP-3 in the baseline variant, this exchange contributes no verification depth to M4 under the rule established in Section 3.5.4, even though the flows are present in the DFD. The only grounded PEP in baseline OP-3 is the Order Service instance that verifies the Buyer role at the TB-1 to TB-3 gateway entry via flows F-10 and F-11. The DDD-aligned variant consequently contributes two grounded PEPs, with the Payment Context PEP performing both a role-verification and an ownership-confirmation round-trip, while the baseline variant contributes a single grounded PEP and one implicit-trust flow. Resulting metric input quantities for OP-3 are shown in Table 5.7.

Table 5.7: Metric input quantities for OP-3.

Quantity	DDD	Baseline
Inter-service delivery points participating in OP-3	2	2
Grounded PEP instances at those delivery points	2	1
Implicit-trust inter-service flows	0	1
Total privileged-operation request flows for OP-3	2	2
Number of containers with auth decision (k)	2	1
Per-operation fragmentation score ($1/k$)	0.50	1.00
Verification depth per grounded PEP (list)	2, 4	2

5.2.4 OP-4: Administrative Dispute Override

OP-4 is the highest-privilege operation in the attacker model, requiring the Administrator role to be verified at both the order-processing and the payment-settlement stages. The flow classification is given in Table 5.8.

Table 5.8: Flow classification for OP-4 (per Table 3.2).

Class	DDD	Baseline
Delivery	F-07, F-14-D	F-07, F-14-D
Verification	F-10, F-11, F-12, F-13	F-10, F-11
Non-inter-service	F-03, F-17, F-18, F-19	F-03, F-17, F-18, F-19

In the DDD-aligned variant, flow F-14-D crosses the TB-4 to TB-5 trust boundary, causing the Payment Context to perform an independent Administrator role verification via flows F-12 and F-13 before admitting the settlement override write to the Payment Data Store, as shown in Figure 4.5. In the baseline variant, F-14-D is an intra-container flow within TB-3, where the Payment Service receives the settlement override request without a trust-boundary crossing event and applies no independent PEP, making that delivery point an implicit-trust flow whose admission relies entirely on the Administrator role check already performed by the Order Service at the TB-1 to TB-3 gateway entry on flow F-07. The resulting metric input quantities are summarized in Table 5.9.

Table 5.9: Metric input quantities for OP-4.

Quantity	DDD	Baseline
Inter-service delivery points participating in OP-4	2	2
Grounded PEP instances at those delivery points	2	1
Implicit-trust inter-service flows	0	1
Total privileged-operation request flows for OP-4	2	2
Number of containers with auth decision (k)	2	1
Per-operation fragmentation score ($1/k$)	0.50	1.00
Verification depth per grounded PEP (list)	2, 2	2

5.3 Aggregate Metric Results

Table 5.10: Aggregated metric values for both architecture variants.

Metric	Property	Direction	DDD	Baseline	Δ (DDD − Base)
M1	Grounded enforcement coverage	↑	1.00	0.57	+0.43
M2	Normalized implicit-trust rate	↓	0.00	0.43	−0.43
M3	Authorization fragmentation score*	↓	0.63	1.00	−0.38
M4	Cross-boundary verification depth*	↓	2.29	2.00	+0.29

M3 and M4 carry no reliable meaning on their own and are read here only in combination with M1. That pairing should be kept in view through the subsections that follow, since both the M3 and the M4 results below turn on whether the containers and checks they count host grounded enforcement, which is exactly what M1 reports.

5.3.1 M1: Grounded Enforcement Coverage

M1 measures what fraction of the privileged inter-service delivery points are protected by an authorization check that the receiving service grounds in state it owns, making it a joint measure of enforcement coverage and quality. It is computed as the ratio of grounded PEP instances to the total number of inter-service delivery points participating in privileged operations, including delivery points at which no PEP of any kind is present, namely

$$\mathrm{M1} = \frac{\sum_{\mathrm{OP}} \text{grounded PEPs}}{\sum_{\mathrm{OP}} \text{inter-service delivery points}} \tag{5.1}$$

where an inter-service delivery point is any receiving service that accepts a privileged request from another service, regardless of whether a trust boundary is drawn at that exchange in the DFD. In the DDD-aligned variant, every inter-service delivery point crosses a drawn trust boundary and carries a grounded PEP at the receiving service, so the numerator and denominator receive identical per-operation contributions. In the baseline variant, the

intra-container flows F-14-C (OP-2), F-14-A (OP-3), and F-14-D (OP-4) each constitute delivery points at which the receiving service accepts a privileged request without an independent authorization check. These delivery points enter the denominator but contribute nothing to the numerator. Substituting the per-operation counts from Tables 5.3–5.9 gives:

$$\mathrm{M1}_{\mathrm{DDD}} = \frac{1+2+2+2}{1+2+2+2} = \frac{7}{7} = 1.00 \tag{5.2}$$

$$\mathrm{M1}_{\mathrm{BASE}} = \frac{1+1+1+1}{1+2+2+2} = \frac{4}{7} \approx 0.57 \tag{5.3}$$

The DDD-aligned variant achieves complete grounded coverage across all seven delivery points, while the baseline variant covers only four of seven, yielding a delta of approximately +0.43. The three uncovered baseline delivery points correspond to the Payment Service instances in OP-2, OP-3, and OP-4, where in each case the receiving service admits the privileged request on the basis of implicit trust in the upstream caller rather than through an independent authorization decision of its own.

5.3.2 M2: Normalized Implicit-Trust Rate

M2 measures what fraction of the privileged inter-service flows reach a receiving service that performs no independent authorization check and so relies on implicit trust in the caller. It is calculated as the ratio of implicit-trust inter-service flows to the total number of privileged-operation request flows across all four operations, namely

$$\mathrm{M2} = \frac{\sum_{\mathrm{OP}} \text{implicit-trust flows}}{\sum_{\mathrm{OP}} \text{request flows}} \tag{5.4}$$

where the denominator includes all privileged-operation request flows regardless of whether they cross a drawn trust boundary, and excludes verification flows, which contribute only to M4. In the DDD-aligned variant, every inter-service request flow crosses a boundary and carries a grounded PEP at its destination, so the implicit-trust numerator is zero across all operations. In the baseline variant, F-14-C (OP-2), F-14-A (OP-3), and F-14-D (OP-4) are all intra-container flows within TB-3 whose receiving service performs no independent authorization check, contributing three implicit-trust

flows out of seven total privileged-operation request flows. Substituting the per-operation counts from Tables 5.3–5.9 gives:

$$\mathrm{M2}_{\mathrm{DDD}} = \frac{0+0+0+0}{1+2+2+2} = \frac{0}{7} = 0.00 \tag{5.5}$$

$$\mathrm{M2}_{\mathrm{BASE}} = \frac{0+1+1+1}{1+2+2+2} = \frac{3}{7} \approx 0.43 \tag{5.6}$$

The DDD-aligned variant records a normalized implicit-trust rate of 0.00 across all seven request flows, while the baseline variant records approximately 0.43, meaning that roughly three in every seven request flows in the baseline deliver a privileged request to a service that applies no independent authorization check. OP-2, OP-3, and OP-4 each contribute one implicit-trust flow, corresponding to the Payment Service receiving an intra-container privileged request (F-14-C, F-14-A, and F-14-D respectively) without an independent PEP at that delivery point.

Within the attacker model defined in Section 3.3.3, a zero implicit-trust rate means that no inter-service privileged request in the DDD-aligned variant structurally depends on a compromised or misconfigured upstream service having already verified the caller's authorization rights before the downstream receiver admits the operation. This directly addresses the lateral movement vulnerability that M2 was defined to capture in Section 3.5.2, where a downstream service accepting a forwarded request without an independent authorization check creates a structural precondition under which an upstream compromise propagates privilege escalation to services that perform no independent verification of their own.

5.3.3 M3: Authorization Fragmentation Score

M3 measures, on average, how concentrated each operation's grounded authorization decisions are within a single boundary container rather than spread across several. It is calculated as the mean, over all four operations, of the reciprocal of k_{OP}, that is the number of distinct boundary containers contributing at least one grounded PEP instance to that operation, namely

$$\mathrm{M3} = \frac{1}{4} \sum_{\mathrm{OP}} \frac{1}{k_{\mathrm{OP}}} \tag{5.7}$$

In the DDD-aligned variant, OP-1 involves only the Catalog Context container (TB-3 Catalog boundary container, k=1), while OP-2, OP-3,

and OP-4 each place grounded PEP instances in two dedicated containers, specifically TB-4 (Order Context) and TB-5 (Payment Context), giving k=2 and a per-operation score of 0.50 for each. In the baseline variant, all grounded PEP instances across every operation reside within the single shared TB-3 container, so k=1 and the per-operation score is 1.00 for all four operations. Substituting the per-operation k values from Tables 5.3–5.9 gives:

$$\mathrm{M3}_{\mathrm{DDD}} = \frac{1}{4}\left(\frac{1}{1} + \frac{1}{2} + \frac{1}{2} + \frac{1}{2}\right) = \frac{1}{4}(2.50) \approx 0.63 \tag{5.8}$$

$$\mathrm{M3}_{\mathrm{BASE}} = \frac{1}{4}\left(\frac{1}{1} + \frac{1}{1} + \frac{1}{1} + \frac{1}{1}\right) = \frac{1}{4}(4.00) = 1.00 \tag{5.9}$$

The DDD-aligned variant scores lower on M3 than the baseline variant (0.63 versus 1.00, delta −0.38). Read jointly with M1 as required by Section 3.5.3, this is the favorable direction, since the lower score indicates that the four operations distribute their grounded authorization decisions across multiple per-context PEP instances rather than concentrating them within a single shared container. Two structurally different conditions can produce different M3 scores, and distinguishing them is the central interpretive obligation for this metric. The first condition is fragmentation resulting from distributed but fully covered enforcement, where each boundary container that contributes an authorization decision hosts a grounded PEP, so that the lower score reflects the number of independent enforcement points rather than any enforcement deficit. In the DDD-aligned variant, M1 = 1.00 confirms that every delivery point carries a grounded PEP, and the lower M3 is a structural consequence of per-context enforcement rather than of missing checks. The second condition is apparent concentration produced by absorbing multiple domain responsibilities into a shared container that relies on implicit trust, so that the higher score is an artifact of boundary collapse rather than of strong co-located enforcement. In the baseline variant, M3 = 1.00 is achieved alongside M2 = 0.43, meaning that the concentration measure captures the absence of inter-container splits rather than the presence of grounded coverage at every delivery point. Interpreting M3 in isolation is therefore insufficient, and M3 must be read jointly with M1 and M2 to distinguish these two structurally opposite conditions.

5.3.4 M4: Cross-Boundary Verification Depth

M4 measures, on average, how many trust-boundary crossings each grounded PEP must traverse to complete its authorization verification. It is computed as the mean verification depth d_i across all grounded PEP instances in a variant, where d_i counts the number of trust-boundary crossings on the round-trip sub-path that the grounded PEP traverses to reach the authoritative identity data store, namely

$$\mathrm{M4} = \frac{\sum_{i=1}^{|\mathcal{P}|} d_i}{|\mathcal{P}|} \tag{5.10}$$

where $\mathcal{P}$ denotes the set of all grounded PEP instances for the variant. In the DDD-aligned variant, six of the seven PEP instances each traverse a single round-trip to the Identity Context (depth 2), while the Payment Context instance in OP-3 additionally traverses an ownership confirmation sub-path to the Order Context via flows F-15 and F-14-B, yielding depth 4 for that instance alone. In the baseline variant, all four PEP instances perform a single round-trip to the Identity Service (depth 2 each). Substituting the per-PEP depth values derived from the verification sub-paths in the per-operation flow diagrams gives:

$$\mathrm{M4}_{\mathrm{DDD}} = \frac{\overbrace{2}^{\text{OP-1}} + \overbrace{2+2}^{\text{OP-2}} + \overbrace{2+4}^{\text{OP-3}} + \overbrace{2+2}^{\text{OP-4}}}{7} = \frac{16}{7} \approx 2.29 \tag{5.11}$$

$$\mathrm{M4}_{\mathrm{BASE}} = \frac{\overbrace{2}^{\text{OP-1}} + \overbrace{2}^{\text{OP-2}} + \overbrace{2}^{\text{OP-3}} + \overbrace{2}^{\text{OP-4}}}{4} = \frac{8}{4} = 2.00 \tag{5.12}$$

The higher M4 in the DDD-aligned variant (2.29 versus 2.00, delta +0.29) is achieved alongside M1 = 1.00 and runs counter to the directional prediction stated in Section 3.5. The bounded-cost analysis below does not retract that prediction outcome, but rather provides the structural explanation that, under the partial-support criterion of Section 3.5, permits the divergence to be classified as a structural cost rather than evidence against the hypothesis itself. Confirming this classification requires the two properties established in Section 3.5.4, and inspection of the per-operation flow diagrams confirms both. Every verification sub-path in the DDD-aligned variant terminates directly at a single authoritative data store, with no intermediate service on

any sub-path, so the maximum sub-path length is two crossings throughout. The depth of four observed at the Payment Context PEP in OP-3 arises because that operation imposes two logically independent authorization conditions on a single enforcing service, namely Buyer role sufficiency and order ownership, each requiring its own sub-path. Hence, it is the authorization logic of OP-3 that produces the depth, not the boundary alignment strategy. The baseline's lower M4 of 2.00 is partly an artifact of enforcement absence, since its three missing grounded PEP instances contribute zero to the numerator while reducing the denominator. The question of at what M4 value verification depth becomes a net liability is not answerable from this single-domain study and is identified in Section 6.3 as a target for multi-domain replication.

5.4 Relation to Prior Work

The M1 and M2 results provide structural evidence for the mechanism that Berardi et al. and Hutasuhut et al. identified as the root condition for access control failures in microservice systems, namely the absence of architectural-level enforcement structures that guarantee consistent authorization across service boundaries [6], [16]. The missing-permission-check and improper-identity-propagation vulnerability categories that Jayalath et al. classify appear directly in the baseline variant, where three inter-service delivery points in OP-2, OP-3, and OP-4 carry privileged requests to services that perform no independent authorization check on the incoming operation [5]. In the DDD-aligned variant, both conditions are eliminated structurally, where every inter-service delivery point constitutes a mandatory trust-boundary crossing at which a grounded PEP instance is required by the DFD topology, so neither a missing check nor an implicit-trust propagation is structurally possible within that decomposition model.

The M1 = 1.00 result in the DDD-aligned variant is a design-level instantiation of the per-service fine-grained authorization that Chandramouli prescribes as structurally distinct from gateway-level filtering, and whose absence constitutes a per-service authorization deficit that the API gateway cannot remedy on its own [3]. The three-layer defense-in-depth architecture that OWASP describes as the recommended enforcement model is produced in the DDD-aligned variant by the boundary placement strategy itself rather than by implementation discipline, because each bounded context boundary in the DFD constitutes a mandatory enforcement point at which a PEP instance is required before the receiving service may admit the operation to its domain logic. In the baseline variant, the TB-3 consolidation removes one of those

layers from the enforcement stack for OP-2, OP-3, and OP-4 without any corresponding mitigation at the design level, producing exactly the structural deficit that both sources identify as a recurring category of microservice authorization weakness [2], [3].

Section 2.4.4 identified that no prior work had directly compared DDD-aligned and non-DDD-aligned microservice architectures on authorization enforcement consistency as a measured structural property. The metric differences in Table 5.10 constitute structured, model-derived comparative evidence for that gap, produced through a controlled comparison in which the two variants differ on exactly the boundary alignment strategy and no other design variable. The direction of those differences is consistent with the theoretical argument, advanced in the background review, that co-location of domain responsibility and authorization responsibility within the same boundary container improves enforcement consistency, and the M1 through M4 framework provides a repeatable instrument by which that argument can be tested against other domain models and decomposition strategies.

5.5 Hypothesis Assessment

The research hypothesis, as stated in Section 3.5, predicts that the DDD-aligned variant will achieve a higher M1 than the baseline variant, a lower M2 than the baseline variant, an equal or lower M4 than the baseline variant, and a lower M3 conditional on the DDD-aligned variant attaining M1 = 1.0 against a strictly lower baseline M1. Each prediction is evaluated independently against the aggregate results in Table 5.10. M1 moves in the predicted direction without qualification, rising from 0.57 in the baseline to 1.00 in the DDD-aligned variant (delta +0.43), which both satisfies the M1 prediction and activates the conditional M3 prediction defined in Section 3.5. M2 moves in the predicted direction without qualification, falling from 0.43 to 0.00 (delta −0.43). With the M1 = 1.0 condition met, the directional prediction for M3 applies, and the observed values (0.63 in the DDD-aligned variant against 1.00 in the baseline, delta −0.38) satisfy that prediction. The lower DDD-aligned M3 reflects authorization decisions placed in separate containers that each host a grounded PEP, while the baseline's higher M3 alongside M2 = 0.43 confirms that its apparent concentration arises from boundary collapse rather than from co-located grounded enforcement. M4 alone diverges from the primary direction of its prediction, with the DDD-aligned value of 2.29 against a baseline value of 2.00 (delta +0.29), the higher value reflects deeper verification paths on fully grounded PEP instances, not reduced enforcement,

and the baseline achieves its lower M4 partly through the suppression of grounded PEP instances that contribute zero to the mean. The bounded-cost analysis in Section 5.3.4 confirms that both structural properties required by Section 3.5.4 hold for the DDD-aligned variant, namely that every verification sub-path terminates at a single authoritative data store and that the depth of each PEP instance is determined by the logical complexity of its authorization requirements rather than by the service count, so the divergence qualifies as the bounded structural cost that Section 3.5.4 defines as acceptable rather than as the compounding verification-chain complexity that a rising M4 would otherwise signal. Three of the four directional predictions are therefore met (M1, M2, and the M1-conditional M3), and the single diverging metric (M4) admits a structural explanation that does not reverse the hypothesis itself. Under the pre-committed criterion of Section 3.5, this constitutes partial support for the hypothesis. The conclusion applies to the specific pair of variants modeled here and should not be read as a general claim about DDD-aligned versus non-DDD-aligned decomposition.

The identical OP-1 scores across both variants function as an internal validity check for the measurement procedure, as discussed in Section 5.2, and support the attribution of the OP-2, OP-3, and OP-4 differences to the boundary alignment strategy rather than to measurement artifacts.

The magnitude of the metric differences, particularly the M1 delta of $+0.43$ and the M2 delta of -0.43, is specific to the four selected privileged operations and to the e-commerce marketplace domain used in this study. The directional result may generalize to other domains, but the specific magnitudes depend on the proportion of cross-context operations and on the number of services consolidated within a shared container in the baseline, both of which are domain and design-specific.

Chapter 6

Conclusions and Future work

This chapter draws the investigation together and reflects on its results. Section 6.1 states the conclusions of the four-metric comparison in Chapter 5 and evaluates them against the research question and hypothesis stated in Chapter 1. Section 6.2 sets out the limitations that bear on how the results should be read. Section 6.3 identifies extensions and open questions that follow from the study, and Section 6.4 closes with reflections on the architectural and ethical implications of the work.

6.1 Conclusions

This study investigated whether aligning microservice boundaries with DDD bounded contexts that encapsulate security responsibilities produces more consistent authorization enforcement than a baseline decomposition that does not apply domain-aligned boundary placement. The four-metric comparison in Chapter 5 and Table 5.10 partially supports the hypothesis under the pre-committed evaluation criterion of Section 3.5, which records partial support when three of the four directional predictions are met and the diverging metric admits a structural explanation that does not reverse the hypothesis itself. M1 satisfies its prediction, with enforcement coverage complete in the DDD-aligned variant and absent at three baseline delivery points. M2 satisfies its prediction, with implicit-trust flows eliminated in the DDD-aligned variant against roughly 43% of inter-service flows in the baseline. The M1 = 1.0 outcome activates the M1-conditional directional prediction for M3, which is also met, since the lower DDD-aligned score reflects distributed but fully grounded enforcement across separate boundary containers rather than missing coverage. M4 alone diverges from the primary direction of its

prediction, since its higher DDD-aligned value reflects deeper verification paths on fully grounded PEP instances rather than reduced enforcement, as required by the joint interpretation in Section 3.5.4, and that structural explanation is consistent with the partial-support threshold rather than at odds with it.

The principal finding for the variant pair modeled in this study is that, for the four privileged operations selected, DDD boundary alignment placed a grounded PEP at every inter-service delivery point (M1 = 1.00) and produced no implicit-trust flows (M2 = 0.00), while the perimeter-trust baseline configuration adopted in Section 4.3.1 left three delivery points uncovered and three flows implicit-trust. The structural mechanism that produces this outcome is the co-location of domain ownership with the enforcing service within each dedicated boundary container, which makes grounded authorization a structural consequence of the decomposition rather than an additional design obligation imposed on top of it. In OP-2, OP-3, and OP-4, the baseline variant fails to achieve these properties because the TB-3 consolidation removes the trust-boundary crossing between the Order Service and the Payment Service, eliminating the structural enforcement point at which the Payment Service would otherwise be required to host a grounded PEP, as documented in Sections 5.2.2, 5.2.3, and 5.2.4. These deltas should therefore be read as the comparison against this particular weak baseline, the perimeter-trust configuration justified in Section 4.3.1, since other non-DDD-aligned configurations that retain per-service enforcement would close part of the observed gap.

DDD alignment distributes authorization decisions across more boundary containers (M3 = 0.63 versus 1.00 in the baseline) and increases the mean verification depth (M4 = 2.29 versus 2.00). As demonstrated in Section 5.3.4, this counter-directional M4 result is a bounded structural cost for the operations studied here, since every verification sub-path terminates at a single authoritative source and the depth of each PEP instance is determined by the logical complexity of its authorization requirements. The point at which additional verification depth becomes a net liability rather than a bounded cost cannot be determined from this single-domain comparison and is an open question for future work. The verification sub-paths identified in the per-operation DFDs represent inter-service dependencies whose reliability implications fall outside the scope of this design-level study.

The study successfully operationalized authorization enforcement consistency as four model-derived metrics applicable to a design-level comparison without a running implementation. The metric framework M1 through M4,

defined in full in Section 3.5, is a methodological contribution independent of the directional hypothesis result that provides a repeatable procedure for evaluating authorization enforcement structure at the DFD level, applicable to any microservice architecture for which a DFD and STRIDE analysis can be constructed.

The practical implication for architects working in security-sensitive microservice systems is to treat bounded contexts not only as domain modularization units but as explicit trust-boundary containers, because this structural choice is what produces complete enforcement coverage and zero implicit-trust flows by design rather than by convention. The associated cost is the additional verification depth in cross-context operations, which must be accounted for in resilience planning for the verification sub-paths that per-context enforcement requires.

6.2 Limitations

The two architecture variants are designed around a single e-commerce marketplace domain with four selected privileged operations. The specific metric delta values observed in this study are a product of the proportion of cross-context operations in this domain and the specific consolidation pattern of the baseline TB-3 container. Domains with a higher proportion of intra-context operations, or with more complex cross-context ownership chains, would produce different relative metric magnitudes, and the directional result, while plausibly general, cannot be assumed to hold in magnitude for other domains or operation sets. The delimitation originally stated in Section 1.6 applies here.

M1 = 1.00 confirms structural enforcement coverage at the design level, meaning that a grounded PEP is structurally present at every inter-service delivery point. It does not guarantee that PEP instances are correctly implemented in deployed code, since a service that technically places a PEP at the boundary but does not correctly consult the relevant domain state would satisfy the metric structurally while remaining insecure in practice. The gap between design-level structural compliance and runtime correctness is outside the method's scope, as acknowledged in Sections 3.4.1 and 3.6.1.

The baseline variant modeled in this study represents one specific non-DDD-aligned configuration, namely the perimeter-trust consolidation justified in Section 4.3.1. It does not represent the space of non-DDD-aligned configurations. A baseline that preserved a separate deployable service per business capability while still drawing boundaries by technical convenience

rather than by bounded context would close part of the M1 and M2 gap without changing the boundary alignment strategy itself. The metric deltas reported in Table 5.10 should therefore be read as the comparison between DDD alignment and this particular weak baseline, not as the comparison between DDD alignment and non-DDD-aligned decomposition in general.

The comparison reported in this study consists of one DDD-aligned variant and one baseline variant, both constructed from the same domain scope, and the four metric values reported in Table 5.10 are deterministic outputs of those two specific DFD models. The baseline TB-3 shared container assignment represents a modeling decisions whose alternative executions would directly affect M3 scores, since a different boundary placement for any of the three co-located services would change the per-operation k values and therefore the fragmentation scores. Moreover, the same researcher who designed both variants also specified the metric definitions and applied the STRIDE-per-element procedure, and although that procedure is deterministic once a DFD is fixed, that alone does not address the prior risk that the metric definitions, scope choices, and modeling judgments were shaped by familiarity with the expected directional result. The single-researcher condition compounds this risk because no independent check existed at the points where the most consequential choices were made, namely the selection of the baseline boundary configuration, the inclusion or exclusion of borderline flows from the trust-boundary set, and the counting rules that govern M1 through M4. The reproducibility claim made in Section 3.6.2 is therefore limited. Consequently, the deltas reported here should be read as the output of one specific design-level instantiation of the comparison rather than as estimates of a population-level effect.

Repudiation, Information Disclosure, and Denial of Service threats were excluded from the STRIDE enumeration, as documented in Section 3.3.5. The M1 through M4 metrics consequently capture authorization enforcement consistency specifically in relation to Elevation of Privilege, Spoofing, and Tampering. Security properties relevant to the excluded threat categories, for example information leakage across context boundaries, are not assessed by the metric framework and would require separate analysis.

The M3 fragmentation score assigns each operation a score of $1/k$, where k is the number of boundary containers contributing an authorization decision. This formulation assumes that distributing authorization decisions across more containers is uniformly worse from an enforcement perspective. The results of this study show that a lower M3 can reflect either an enforcement deficit, namely authorization distributed without full coverage, or a structural

necessity, namely independent enforcement at each boundary with full coverage as in the DDD-aligned variant, and the $1/k$ formulation does not distinguish these two conditions intrinsically. Correct interpretation of M3 therefore requires joint reading with M1, and this is a known limitation of the current metric design rather than a property that can be resolved by additional data.

A further limitation concerns the unweighted, count-based construction of all four metrics. Each privileged operation and each inter-service delivery point contributes equally to M1 through M4, irrespective of the real-world consequences of an enforcement gap at that point. An uncovered delivery point belonging to a low-impact operation such as the listing publication of OP-1 lowers M1 by the same amount as an uncovered delivery point belonging to the irreversible and financially significant dispute override of OP-4, as set out in Table 4.1, even though a missing grounded PEP at the latter carries far graver security consequences. The metrics therefore quantify the structural consistency of authorization enforcement across the modeled flows, but they do not express the severity-weighted risk exposure of the gaps they reveal, and a variant could in principle attain an acceptable aggregate M1 while leaving its most consequential operation uncovered. This is a property of the metric construction rather than of the particular models compared here, and it bounds how directly the aggregate scores can be read as a measure of residual risk.

6.3 Future work

Several directions for future work follow from the study, ordered below by their proximity to the central empirical question the thesis leaves open. Runtime implementation and testing of both architecture variants is the most direct extension. The DFD models and metric definitions produced in this thesis provide sufficient structural specification to guide implementation, and the central empirical question left open is whether the design-level metric differences, particularly the M1 and M2 deltas, translate into measurable runtime vulnerability differences under controlled penetration testing or automated authorization boundary testing. Runtime testing was considered and rejected as out of scope for this study for the reasons documented in Section 3.4.1, and it remains the natural next validation step.

Application of the same metric framework to domains with structurally different authorization topologies is the next step toward establishing generalizability beyond the single-domain limitation identified in Section 6.2. Two candidate domain types with distinct authorization characteristics

illustrate the range of the open question. A domain with attribute-based access control and cross-context data sensitivity, such as a clinical healthcare records system, would test how M1 and M2 behave when authorization conditions are continuous rather than binary role assertions. A domain with regulatory approval chains and multi-party authorization, such as financial services settlement, would test whether longer verification chains in the DDD-aligned variant produce M4 values that materially affect the hypothesis direction.

A refined M3 variant that weights the per-operation fragmentation score by per-container enforcement coverage is the most directly motivated metric-level next step. The current formulation does not intrinsically distinguish between coverage-supported fragmentation, where each container that contributes a decision hosts a grounded PEP, and coverage-absent fragmentation, where decisions are distributed across containers some of which lack grounded coverage. A coverage-weighted variant would make the distinction intrinsic to the metric rather than requiring post-hoc joint reading with M1, addressing the M3 limitation discussed in Section 6.2 at the level of the metric definition itself.

An automated tool-chain that computes M1 through M4 from a machine-readable DFD would make the framework accessible to practitioners operating under time constraints who cannot perform the manual enumeration procedure at scale, and would eliminate the single-researcher reliability gap discussed in Section 6.2 by replacing interpretive modeling judgments with a deterministic parsing of the DFD element and container structure. The metric definitions in Section 3.5 are stated with sufficient precision to support such an implementation directly.

Application of solution-aware DFD notation, as proposed by Sion et al., is a natural extension that was deliberately excluded in this study for the reasons documented in Section 3.4.3. The specific research question it would address is how mitigation density and distribution interact with the boundary alignment strategy metrics, and whether annotating mitigations in the DFD shifts the relative M1 through M4 scores in ways that affect the hypothesis outcome. This extension would also make the framework applicable to architectures where the security solution layer is already specified and cannot be abstracted away without losing analytical fidelity [8].

6.4 Reflections

This study shows that a design-level architectural choice, specifically the strategy by which trust boundary containers are assigned to services, has

measurable consequences for authorization enforcement consistency that are assessable before any implementation cost is incurred. The implication is that security-relevant structural decisions can be evaluated and iterated at the threat modeling stage, at a point where the cost of changing a boundary placement is the cost of redrawing a DFD container rather than the cost of restructuring deployed services and their authorization logic. The four-operation comparison in this thesis is a bounded demonstration of this property rather than a general proof, and claims about its applicability beyond the studied domain and operation set should be made with corresponding caution.

The baseline variant's implicit-trust flows, identified in OP-2, OP-3, and OP-4, create structural preconditions under which a lateral movement scenario could result in unauthorized financial operations, should those flows be exploited in a deployed system that instantiates the baseline design. This study identifies that structural risk condition at the design level and does not demonstrate a proven or exploitable vulnerability in any deployed system. The ethical implication is that software architects carry a responsibility to address authorization enforcement by structural design rather than by convention or by the assumption that an upstream service has already verified the caller's access rights, because the latter assumption is invisible in a deployed system until it is exploited. The metric framework proposed in this thesis offers one instrument for making that assumption visible and measurable at the stage where correcting it is least costly.

The societal contribution of this work is to support architectural decisions that reduce the future likelihood of authorization failures leading to unauthorized access to user data or privileged system operations. The cost of correcting such failures is highest when they are discovered in deployed systems, since the affected users are real and the data has often already been exposed. A design-level method that makes the structural preconditions for those failures visible at the threat modeling stage shifts the point of correction earlier, before users are exposed [42].

Across the three dimensions of sustainability, the social contribution is the reduced likelihood of data-exposure harm to real users discussed above, the economic contribution is the lower barrier to security analysis for teams without dedicated specialists and the earlier, cheaper point of correction, and finally the ecological dimension is not comparatively meaningful here, since a design-level modeling method and theoretical analysis has no runtime footprint and no resource consumption beyond document preparation.

The methodology proposed here does not require a running system, specialized tooling, or access to production data, and can be applied by a

small team using a threat modeling tool like OWASP Threat Dragon. This lowers the barrier to security analysis for teams that lack dedicated security infrastructure, including teams in early-stage projects and in organizations without security specialists on staff. The framework does not replace the work of those specialists where they are available, nor does it offer the same credibility, but it provides a structured starting point where they are not.

References

[1] N. Dragoni et al., "Microservices: Yesterday, today, and tomorrow," *CoRR*, vol. abs/1606.04036, 2016. arXiv: 1606.04036. [Online]. Available: http://arxiv.org/abs/1606.04036.

[2] OWASP, *Microservices security cheat sheet*, OWASP Cheat Sheet Series. Available online, 2025. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Microservices_Security_Cheat_Sheet.html.

[3] R. Chandramouli, "Security strategies for microservices-based application systems," National Institute of Standards and Technology (NIST), Tech. Rep. Special Publication 800-204, 2019. DOI: 10.6028/NIST.SP.800-204. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-204.

[4] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2004, ISBN: 0-321-12521-5.

[5] R. K. Jayalath, H. Ahmad, D. Goel, M. S. Syed, and F. Ullah, "Microservice vulnerability analysis: A literature review with empirical insights," *IEEE Access*, 2024, arXiv:2408.03960v1. DOI: 10.48550/arXiv.2408.03960.

[6] D. Berardi, S. Giallorenzo, J. Mauro, A. Melis, F. Montesi, and M. Prandini, "Microservice security: A systematic literature review," *PeerJ Computer Science*, vol. 8, e779, 2022. DOI: 10.7717/peerj-cs.779.

[7] B. Hippchen, P. Giessler, R. H. Steinegger, M. Schneider, and S. Abeck, "Designing microservice-based applications by using a domain-driven design approach," *International Journal on Advances in Software*, vol. 10, no. 3 & 4, pp. 432–445, 2017.

[8] L. Sion, K. Yskout, D. Van Landuyt, and W. Joosen, "Solution-aware data flow diagrams for security threat modeling," in *Proceedings of the 33rd Annual ACM Symposium on Applied Computing (SAC '18)*, ACM, 2018, pp. 1426–1435. DOI: 10.1145/3167132.3167285.

[9] A. Shostack, "Experiences threat modeling at Microsoft," in *Proceedings of the Workshop on Modelling Security (ModSec08)*, 2008. [Online]. Available: https://shostack.org/files/papers/modsec08/Shostack-ModSec08-Experiences-Threat-Modeling-At-Microsoft.pdf.

[10] M. I. Hossain, N. M. I. Chowdhury, and R. Hasan, "How secure is AI-based coding? A security analysis using STRIDE and data flow diagrams," in *2023 IEEE Virtual Conference on Communications (VCC)*, 2023, pp. 56–61. DOI: 10.1109/VCC60689.2023.10474718.

[11] R. Khan, K. McLaughlin, D. Laverty, and S. Sezer, "STRIDE-based threat modeling for cyber-physical systems," in *2017 IEEE PES Innovative Smart Grid Technologies Conference Europe (ISGT-Europe)*, IEEE, 2017. DOI: 10.1109/ISGTEurope.2017.8260283.

[12] J. Brannen, "Combining qualitative and quantitative approaches: An overview," *Mixing methods: Qualitative and quantitative research*, pp. 3–37, 2017.

[13] A. Tashakkori and C. Teddlie, *Mixed methodology: Combining qualitative and quantitative approaches*. sage, 1998, vol. 46.

[14] P. Merson and J. Yoder, "Modeling microservices with ddd," in *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*, IEEE, 2020, pp. 7–8.

[15] R. H. Steinegger, P. Giessler, B. Hippchen, and S. Abeck, "Overview of a domain-driven design approach to build microservice-based applications," in *The Thrid Int. Conf. on Advances and Trends in Software Engineering*, 2017.

[16] N. R. P. Hutasuhut, M. G. Amri, and R. F. Aji, "Security gap in microservices: A systematic literature review," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 12, pp. 165–170, 2024. DOI: 10.14569/IJACSA.2024.0151218.

[17] P. Billawa, A. B. Tukaram, N. E. D. Ferreyra, J. Steghöfer, R. Scandariato, and G. Simhandl, "Security of microservice applications: A practitioners' perspective on challenges and best practices," *CoRR*, vol. abs/2202.01612, 2022. arXiv: 2202.01612. [Online]. Available: https://arxiv.org/abs/2202.01612.

[18] J. A. Shonubi, "Multi-layered zero trust architectures for cross-domain data protection in federated enterprise networks and high-risk operational environments," *Int. J. Res. Publ. Rev*, vol. 6, pp. 146–169, 2025.

[19] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, "A survey on zero trust architecture: Challenges and future trends," *Wireless Communications and Mobile Computing*, vol. 2022, no. 1, p. 6 476 274, 2022.

[20] V. C. Hu et al., "Guide to attribute based access control (abac) definition and considerations," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-162, Jan. 2014. DOI: 10.6028/NIST.SP.800-162. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-162.

[21] M. B. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," Internet Engineering Task Force (IETF), RFC 7519, May 2015, Standards Track RFC. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[22] D. (Hardt, "The oauth 2.0 authorization framework," Internet Engineering Task Force (IETF), RFC 6749, Oct. 2012, Obsoletes RFC 5849. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749.

[23] L. Leines-Vite, J. C. Pérez-Arriaga, and X. Limón, "Information and communication security mechanisms for microservices-based systems," *International Journal of Network Security & Its Applications (IJNSA)*, vol. 13, no. 6, pp. 85–103, 2021. DOI: 10.5121/ijnsa.2021.13607.

[24] M. G. de Almeida and E. D. Canedo, "Authentication and authorization in microservices architecture: A systematic literature review," *Applied Sciences*, vol. 12, no. 6, p. 3023, 2022. DOI: 10.3390/app12063023.

[25] P. K. Manadhata and J. M. Wing, "An attack surface metric," *IEEE Transactions on Software Engineering*, vol. 37, no. 3, pp. 371–386, 2011. DOI: 10.1109/TSE.2010.60.

[26] OWASP, *Attack surface analysis cheat sheet*, OWASP Cheat Sheet Series. Available online, 2025. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.html.

[27] A. Shostack, *Threat modeling: Designing for security*. John Wiley & Sons, 2014, ISBN: 978-1-118-80999-0.

[28] M. Howard and S. Lipner, *The Security Development Lifecycle*. Jun. 2006, vol. 34, ISBN: 0735622140. DOI: 10.1007/s11623-010-0021-7.

[29] S. Ghasemshirazi, G. Shirvani, and M. A. Alipour, *Zero trust: Applications, challenges, and opportunities*, 2023. arXiv: 2309.03582 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2309.03582.

[30] J. W. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 4th. SAGE Publications, Inc., Jan. 2014, ISBN: 978-1-4522-2610-1.

[31] M. Casula, N. Rangarajan, and P. Shields, "The potential of working hypotheses for deductive exploratory research," *Quality & Quantity*, vol. 55, no. 5, pp. 1703–1725, 2021, ISSN: 1573-7845. DOI: 10.1007/s11135-020-01072-9. [Online]. Available: https://doi.org/10.1007/s11135-020-01072-9.

[32] D. T. Campbell and J. C. Stanley, *Experimental and quasi-experimental designs for research*. Ravenio books, 2015.

[33] L. Sion, K. Yskout, D. Van Landuyt, and W. Joosen, "Risk-based design security analysis," in *Proceedings of the 1st International Workshop on Security Awareness from Design to Deployment*, 2018, pp. 11–18.

[34] T. Milić, Z. Stojanov, and I. Vecštejn, "Application of the stride framework in a microservice application for electronic business," *Journal of Engineering Management and Competitiveness*, vol. 15, pp. 100–109, Dec. 2025. DOI: 10.5937/JEMC2502100M.

[35] A. Venčkauskas, D. Kukta, Š. Grigaliūnas, and R. Brūzgienė, "Enhancing microservices security with token-based access control method," *Sensors*, vol. 23, no. 6, p. 3363, 2023.

[36] D. Landuyt, L. Pasquale, L. Sion, and W. Joosen, "Threat modeling at run time: The case for reflective and adaptive threat management (nier track)," May 2021, pp. 203–209. DOI: 10.1109/SEAMS51251.2021.00034.

[37] P. Roberts and H. Priest, "Reliability and validity in research.," *Nursing standard*, vol. 20, no. 44, pp. 41–46, 2006.

[38] N. Golafshani et al., "Understanding reliability and validity in qualitative research," *The qualitative report*, vol. 8, no. 4, pp. 597–607, 2003.

[39] E. Freudenthal, T. Pesin, L. Port, E. Keenan, and V. Karamcheti, "Drbac: Distributed role-based access control for dynamic coalition environments," in *Proceedings 22nd International Conference on Distributed Computing Systems*, IEEE, 2002, pp. 411–420.

[40] D. Ferraiolo, R. Sandhu, S. Gavrila, D. Kuhn, and R. Chandramouli, "Proposed nist standard for role based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, pp. 224–274, Aug. 2001. DOI: 10.1145/501978.501980.

[41] C. Richardson, *Microservices patterns: with examples in Java*. Simon and Schuster, 2018.

[42] M. Souppaya, K. Scarfone, and D. Dodson, "Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-218, Feb. 2022. DOI: 10.6028/NIST.SP.800-218. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-218.

TRITA-EECS-EX-2026:339
Stockholm, Sweden 2026

www.kth.se